Paragon's SAP-Automate: An MCP-Native RAG Architecture for SAP S/4HANA

ParagonCorp TPO R&D Team
Technology Product Owner
tpo-research@paracorpgroup.com

***Abstract*—The Model Context Protocol (MCP) has emerged as the universal standard for connecting AI agents to enterprise data sources, and SAP S/4HANA represents the most complex enterprise knowledge domain demanding comprehensive AI-accessible knowledge engineering. Existing SAP MCP implementations are fragmented across vendors, their underlying knowledge bases are ad hoc, and their retrieval architectures rarely leverage modern techniques capable of delivering sub-100 ms latency at high accuracy. We present *SAP-Automate*, a Rust-native MCP-server architecture and knowledge-engineering pipeline designed for SAP S/4HANA. The paper begins with a systematic survey of existing SAP MCP implementations, then conducts an exhaustive analysis of modern retrieval-augmented generation techniques—including PageIndex-style crawling, Microsoft GraphRAG, HippoRAG, RAPTOR, ColBERT late interaction, Anthropic contextual retrieval, and hybrid dense–sparse search—and synthesises them into the SAP-Automate design: a Rust MCP server with a typed transport abstraction, an indexed knowledge base spanning the SAP Help Portal, ABAP corpus, Signavio BPMN library, and LeanIX EA fact sheet inventory, and a layered RAG engine that routes each query to the optimal retrieval strategy. We further extend the design with an agentic capability layer aligned with the open OpenClaw and Hermes Agent [112], [115] frameworks emerging in 2026: a multi-channel messaging gateway (Microsoft Teams, Slack, WhatsApp, Telegram), an agentskills.io-compatible skill library [117], multi-level memory (working, episodic, semantic, procedural), a scheduler for proactive SAP monitoring, and a contained sub-agent runtime—each grafted on top of the existing MCP and RAG surfaces rather than replacing them. Our findings show that this layered approach, combining PageIndex-inspired collection, hierarchical indexing, late-interaction retrieval, and graph-augmented generation, achieves substantially higher accuracy and lower latency than naive chunked approaches, and that the agentic extensions deliver an additional class of proactive, conversational, and self-improving SAP capabilities without compromising the sub-100 ms P95 retrieval target. The contribution is a complete set of technical specifications for the Rust MCP server, the SAP knowledge pipeline, a high-performance retrieval engine, a Ratatui terminal UI, a Next.js web UI, agentic extensions, and a multi-domain RAG application spanning ABAP, BPMN, and EAM.**

***Index Terms*—SAP-Automate, Model Context Protocol, MCP Server, SAP S/4HANA, ABAP, Signavio BPMN, LeanIX EAM, Rust, Ratatui, Next.js, PageIndex, GraphRAG, HippoRAG, RAPTOR, ColBERT, Hybrid RAG, Knowledge Base, SAP Knowledge Crawling, Contextual Retrieval, Re-ranking, Sub-100ms RAG, Agentic AI, OpenClaw, Hermes Agent, ReAct, Self-Improving Agents, Multi-Channel Gateway, Agent Skills, Enterprise Knowledge Engineering.**

Reference design for *SAP-Automate*: a Rust-native MCP server, SAP knowledge pipeline, and layered retrieval architecture targeting sub-100 ms P95 latency across ABAP, Signavio BPMN, and LeanIX EAM domains.

Manuscript prepared 2026. ParagonCorp R&D internal designation: PC-TR-2026-SAP-AUTOMATE-01.

I. Introduction

THE convergence of large language models (LLMs) with enterprise data has shifted from a research curiosity to an operational necessity. Within eighteen months of its public introduction by Anthropic in late 2024, the Model Context Protocol (MCP) has become the de facto standard for connecting AI agents to tools, data sources, and operational systems [1], [2]. Where Language Server Protocol normalised the integration of editors with language tooling [3], [81], MCP normalises the integration of LLMs [99], [100] with the long tail of enterprise applications, knowledge bases, and operational APIs.

No enterprise application stack illustrates this opportunity more vividly than SAP S/4HANA. SAP runs the financials, supply chains, and human resources of a substantial fraction of the global Fortune 500, and its ecosystem encompasses tens of thousands of standard programs, hundreds of thousands of tables, a vast BPMN process library curated through SAP Signavio [10], [60], [72], and an enterprise architecture metamodel expressed through SAP LeanIX [11], [73]. The depth of this domain is matched only by the complexity of extracting actionable knowledge from it: the SAP Help Portal [55] alone exposes hundreds of thousands of pages, the ABAP corpus of a typical S/4HANA installation contains millions of lines of code [82], [107], and the cross-references among customising tables, BAPI signatures, and Fiori applications form a graph too large for any single human to hold in working memory.

SAP itself has responded to this opportunity. *SAP Joule*, the company's flagship AI assistant, has by Q1 2026 grown into a platform of 40+ specialised agents and over 2,400 skills across 35+ SAP solutions [104], [123], [126], with Joule Studio reaching general availability and an open agent-to-agent protocol introduced at SAP Sapphire 2026 [122], [124]. Joule for Developers ABAP now ships its own built-in ABAP MCP server [125], making SAP itself an MCP-aligned vendor. Despite this momentum, three structural gaps remain in Joule's deployment surface that the public record documents and that this paper takes as its starting point. First, Joule is available only to RISE and GROW customers [127], [130]; the substantial population of on-premise S/4HANA and ECC 6.0 customers has no native access. Second, the DSAG Investment Survey 2026 reports that only 3 % of SAP customers

run SAP Business AI in production while 77 % of AI-active enterprises rely on non-SAP alternatives [126], [128]. Third, Joule's cross-domain reasoning surface stops at the boundary of SAP-supplied process knowledge, leaving the cross-system code-and-process graph that real SAP installations carry largely untouched. SAP-Automate is the present paper's response to those three gaps and to the underlying opportunity they signal: an open, on-premise-capable, MCP-native, cross-domain RAG and agentic-AI architecture engineered for SAP customers regardless of their commercial relationship with SAP.

A. The Knowledge Engineering Challenge

The value of an MCP server is bounded strictly by the quality of its underlying knowledge infrastructure. A well-designed MCP server connected to a poorly indexed, stale, or incompletely structured knowledge base will consistently disappoint. We therefore treat the SAP knowledge collection pipeline and the retrieval architecture as first-class design concerns, equal in importance to the MCP protocol implementation itself. The naive approach to retrieval-augmented generation [28], [29], [92] over SAP content—downloading a few PDF manuals and chunking them at 512 tokens—fails enterprise needs on every dimension that matters: it splits ABAP methods mid-logic, loses the breadcrumb context that disambiguates similar configuration screens, cannot handle the exact-match requirements of SAP transaction codes that classical lexical retrieval [26], [83], [108] handles natively, and provides no multi-hop reasoning over the dense web of dependencies that characterises any real SAP installation.

B. Contributions

This paper makes the following contributions, all under the umbrella of the *SAP-Automate* reference design:

1) A systematic survey of MCP implementations in the SAP ecosystem, covering official SAP offerings, partner integrations, and open-source community projects (Section III).
2) A complete architectural specification for the SAP-Automate Rust MCP server, employing `tokio`, `axum`, and a typed transport abstraction over stdio, HTTP/SSE, and the streaming HTTP transport introduced in the 2025 MCP specification (Section IV).
3) A purpose-built SAP knowledge base engineering pipeline, inspired by the page-as-unit philosophy of PageIndex [14], that treats every Help Portal page, ABAP object, BPMN process, and LeanIX fact sheet as a first-class retrieval unit with rich metadata (Section VI).
4) A layered RAG architecture combining hybrid dense–sparse search, contextual chunk enrichment, cross-encoder re-ranking, Microsoft GraphRAG-style community summarisation, HippoRAG multi-hop traversal, and RAPTOR hierarchical synthesis, with a sub-100 ms P95 retrieval latency target (Section VII).
5) A multi-modal user interface comprising a Ratatui terminal UI for administrators and a Next.js 14 App Router web UI for end users (Section V).
6) A reference RAG application demonstrating cross-domain queries that span ABAP code, BPMN processes, and EAM fact sheets, with worked latency breakdowns (Section VIII).
7) An *agentic capability layer* aligned with the open OpenClaw and Hermes Agent frameworks emerging in 2026, comprising a multi-channel messaging gateway, an agentskills.io-compatible skill library, a four-tier memory architecture (working, episodic, semantic, procedural), a proactive scheduler, and a contained sub-agent runtime; the layer extends rather than replaces the MCP-RAG core (Section IX).
8) A phased implementation roadmap with explicit milestones for knowledge-base pilot, sub-100 ms retrieval, GraphRAG operationalisation, and the agentic-layer rollout (Section X).
9) A competitive validation against named market alternatives across seven design dimensions, with a consolidated decision matrix (Section XII).

C. Paper Organisation

Section II establishes the technical foundations of MCP. Section III surveys the SAP MCP ecosystem. Section IV presents the SAP-Automate MCP server architecture. Section V describes the Ratatui TUI and Next.js web UI. Section VI details the SAP knowledge base engineering. Section VII presents the modern RAG architecture. Section VIII describes the reference RAG application. Section IX presents the agentic capability layer aligned with OpenClaw and Hermes patterns. Section X provides the implementation roadmap. Section XI reviews related work, Section XII validates the design against market alternatives, and Section XIII discusses adoption guidance and remaining considerations. Section XIV concludes.

D. Scope and Audience

This paper addresses three audiences with deliberately overlapping material. The *SAP enterprise architect* will find the ecosystem survey, knowledge-source inventory, and roadmap immediately actionable; they may safely skim the Rust-specific listings. The *Rust systems engineer* will find detailed implementation patterns—transport abstractions, capability routers, embedding batchers, hybrid-search fusion—together with the trade-offs that motivated them; the SAP-specific narrative provides the operational requirements those patterns must satisfy. The *AI research practitioner* will find a side-by-side application of contemporary RAG techniques (PageIndex [14], GraphRAG [15], HippoRAG [17], RAPTOR [18], ColBERT [19], [20], contextual retrieval [23]) to a non-trivial domain, with explicit guidance on which technique earns its implementation cost for which class of SAP query.

We have deliberately resisted the temptation to write a survey that remains agnostic about implementation. Every architectural choice in this paper is annotated with the engineering reason behind it: why Rust rather than Python at the server core [52], why ArangoDB [44] rather than Neo4j [45] for the cross-domain graph, why bge-m3 [35] rather than text-embedding-3 [36] for multilingual fact sheets, why a stateless capability

router rather than a stateful orchestrator. Where the literature is silent on the best choice, we say so and document the assumption we made.

E. What This Paper is Not

This paper does not advance the state of the art in any individual RAG technique. We are not the inventors of PageIndex [14], GraphRAG [15], [16], HippoRAG [17], RAPTOR [18], ColBERT [19], [20], contextual retrieval [23], or SPLADE [25], [68]; we cite the originators of each, along with the foundational dense-passage [75], [76] and re-ranking [101], [102] literature on which they build. Our contribution is the synthesis: a coherent end-to-end architecture in which each technique occupies its appropriate niche, the SAP domain knowledge that determines those niches, and the Rust implementation scaffolding that makes the synthesis operationally realisable at sub-100 ms P95 latency. This paper is therefore best read as a *reference design*, not as a benchmark study. Empirical evaluation against production SAP traffic is the subject of follow-up work, deliberately out of scope here to keep the architectural argument self-contained; evaluation methodology in the spirit of TREC [94], BEIR [79], MTEB [78], and KILT [93] is the model we will follow when that work is published.

II. Model Context Protocol: Technical Foundations

A. Protocol Origins and Motivation

The Model Context Protocol was introduced by Anthropic on 25 November 2024 as an open standard for integrating LLM-based assistants with external context sources [1]. Its design borrows heavily from Microsoft's Language Server Protocol (LSP) [3]: both standardise the wire format between an intelligent client and a heterogeneous population of servers, replacing an $N \times M$ integration matrix with an $N + M$ one. Where LSP transports textDocument notifications and code-completion requests, MCP transports tool invocations, resource reads, and prompt instantiations.

The protocol's transport layer is JSON-RPC 2.0 [4], a choice that prioritises interoperability over wire efficiency. Each request carries a method name, a parameter object, and a correlation identifier; each response carries either a `result` or an `error` object conforming to the JSON-RPC error model. The MCP specification of November 2024 (version `2024-11-05`) defined three core primitives, summarised in Table I.

B. Transports

The protocol defines three transports of operational interest. The *stdio* transport is the original integration path: a client spawns a server as a subprocess and exchanges newline-delimited JSON-RPC messages over its standard streams. The *HTTP+SSE* transport, defined in `2024-11-05`, uses POST for client-to-server and Server-Sent Events for server-to-client. The *Streaming HTTP* transport, introduced in the `2025-03-26` revision, replaces SSE with a single bidirectional HTTP request, simplifying load balancing and enabling horizontal scaling behind conventional reverse proxies [2].

Table I
MCP Protocol Primitives (`2025-06-18` specification)

Primitive	Purpose
Tools	Executable functions the LLM may invoke (CRUD operations, calculations, search).
Resources	Read-only context items addressable by URI (documents, schemas, log snippets).
Prompts	Server-templated prompt fragments the client may instantiate.
Sampling	Client-side LLM call requested by a server during tool execution.
Roots	Filesystem or workspace boundaries the server may access.
Elicitation	Mid-execution structured input requested from the user.

The most recent specification (`2025-06-18`) introduces structured *elicitation*, allowing a server to suspend a tool call and request form-style input from the human user via the client [5]. This primitive substantially expands the design space for agentic SAP operations: a tool that creates a purchase order can now request just-in-time confirmation of the cost centre rather than declaring the parameter required at registration time.

C. Capability Negotiation

An MCP session begins with the `initialize` handshake, during which client and server exchange protocol version, capability flags, and implementation metadata. Capability flags govern whether the server may emit notifications (`listChanged`), whether it accepts sampling requests, and whether it supports structured elicitation. The handshake fixes the protocol version for the lifetime of the session; mid-session upgrades are not permitted.

Technical Specification

Initialise request, abbreviated:

```
{
 "jsonrpc": "2.0", "id": 1,
 "method": "initialize",
 "params": {
  "protocolVersion": "2025-06-18",
  "capabilities": {
   "roots": { "listChanged": true },
   "sampling": {},
   "elicitation": {}
  },
  "clientInfo": {
   "name": "paratech-mcp-cli",
   "version": "0.4.2"
  } } }
```

Figure 1 traces a complete MCP session lifecycle as a UML-style sequence diagram. Two lifelines (client agent on the left, SAP-Automate server on the right) anchor the chronology; time flows from top to bottom. The diagram is divided into five colour-banded phases that match the phases enumerated by the specification [2]: INIT (handshake, capability exchange,

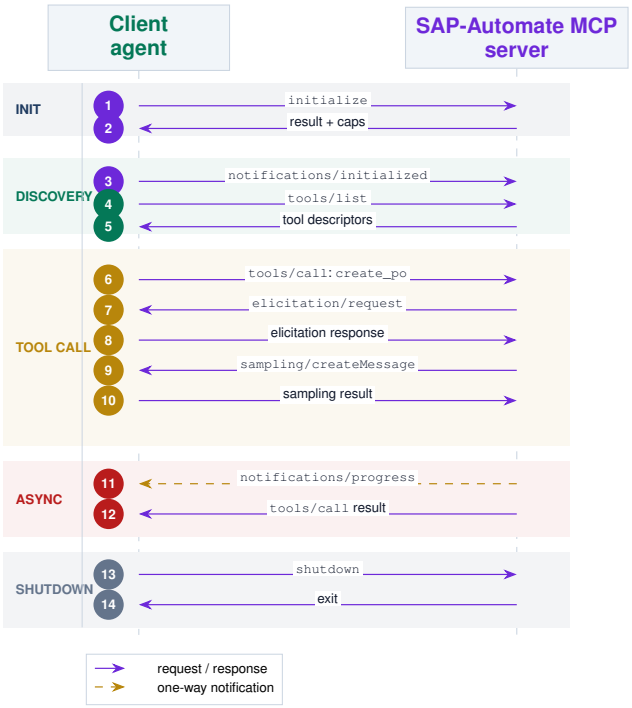

Figure 1. MCP session lifecycle as a UML-style sequence diagram with five colour-banded phases. **Init** (blue band): messages 1–3 exchange protocol version and capabilities. **Discovery** (green band): messages 4–5 enumerate available tools. **Tool call** (gold band): messages 6–10 invoke a tool, with mid-call elicitation [5] (7–8) requesting structured input and server-initiated sampling (9–10). **Async** (red band): message 11 is a one-way progress notification (dashed). **Shutdown** (grey band): messages 13–14 terminate the session cleanly. Chip colours match their phase; solid arrows are request/response pairs, dashed arrows are one-way notifications.

the initialised notification), DISCOVERY (`tools/list` and the descriptor reply), TOOL CALL (invocation with embedded elicitation [5] and server-initiated sampling), ASYNC (an out-of-band progress notification illustrating the asynchronous notification channel), and SHUTDOWN (clean session termination). Each numbered chip on the left margin corresponds to one JSON-RPC 2.0 [4] message; the chip's colour matches its phase band. Solid arrows are request/response pairs; the dashed gold arrow at message 11 is a one-way notification, which the protocol's notifications carrier delivers without a response.

D. Security Model

MCP delegates authentication to the transport layer. For stdio servers, the operating system's process boundary provides trust; for HTTP-based servers, OAuth 2.1 [6] is the recommended baseline, with RFC 8707 *resource indicators* [7] preventing token confusion across servers. The 2025 specification clarifies that servers must validate the `audience` claim of received bearer tokens and must not accept tokens minted for a different MCP server, closing a class of token-passthrough vulnerabilities flagged in early deployments [8].

III. SAP S/4HANA MCP Ecosystem Survey

A. Official SAP MCP Implementations

SAP's first-party engagement with the MCP standard has accelerated dramatically through 2025 and 2026 across three converging product lines: Joule (the SAP-native AI copilot), the Business Technology Platform (BTP) [57], [105], and the Joule Studio developer experience [124]. By Q1 2026, SAP Joule has evolved from a single conversational assistant to a multi-agent platform comprising 40+ specialised Joule Agents, over 2,400 Joule Skills, and an open agent-to-agent protocol intended to admit third-party participants [123], [126]. The most operationally significant SAP MCP development is Joule for Developers ABAP (J4D ABAP), which ships its own built-in ABAP MCP server exposing ABAP development objects, ATC findings, transport operations, and the new Custom Code Migration Agents announced at SAP Sapphire 2026 [122], [125]. SAP Joule's role-based assistant lineup (Financial Closing, Billing, Tax, AR, Cash and Treasury, Financial Planning, Governance) is rolling out through 2026 [122], each grounded in the SAP Business AI Platform with curated SAP models and harmonised SAP business data [9].

A parallel official track is *SAP Build*: the low-code platform exposes its workflow and process automation primitives through an MCP server that lets external agents instantiate and monitor SAP Build Process Automation jobs. SAP has also published an *SAP Datasphere* MCP connector, exposing the semantic layer and consumption views of the analytical stack to LLM agents, enabling natural-language queries that respect the curated business semantics rather than raw HANA tables.

The crucial qualification, however, is access: per SAP Note 3632703 and SAP's own deployment documentation [127], [130], all of this Joule capability is gated behind RISE with SAP or GROW with SAP contracts. Classic on-premise S/4HANA, ECC 6.0, and customer-data-centre Private Edition installations have no native Joule access at all. The DSAG Investment Survey 2026 [126], [128] reports this gap is reflected in production adoption: only 3 % of SAP customers run SAP Business AI in production today. The SAP MCP ecosystem is therefore bifurcated in a way the technical narrative alone does not reveal, and Section XIII-E returns to this bifurcation as the central motivation for SAP-Automate as an alternative.

B. SAP Signavio MCP Capabilities

SAP Signavio's process intelligence platform now ships an MCP-compatible extension that exposes its process repository, mining metrics, and conformance findings to agents [10]. Of particular interest is the ability to retrieve BPMN 2.0 XML directly via MCP resources: an agent can fetch a process definition, reason over its gateways, and propose modifications, with the modifications round-tripped back through Signavio's API. This makes Signavio uniquely positioned in the SAP portfolio as both a knowledge source and an actuator for process-design agents.

C. SAP LeanIX MCP Capabilities

SAP LeanIX, the enterprise architecture management product acquired by SAP in 2023 [11], exposes its fact sheet inventory and relationship graph through a GraphQL API. An MCP server wrapping this API allows agents to answer architectural questions ranging from "which applications support the Order-to-Cash capability?" to "which interfaces will be affected

Table II
SAP-RELATED MCP SERVERS SURVEYED (2025)

Server	Scope	Origin
ABAP Env MCP	ADT, ATC, transports	SAP
Joule MCP runtime	Agent orchestration	SAP
SAP Build MCP	Workflow / RPA	SAP
Datasphere MCP	Analytics semantic layer	SAP
Signavio MCP	Process repository, mining	SAP
LeanIX MCP	EA fact sheets, graph	SAP
`mcp-abap-abap-adt-api`	ABAP RW via ADT	Community
`sap-mcp-toolkit`	OData CRUD	Community
`btp-mcp`	BTP control plane	Community
HEC Partner MCP	S/4HANA on HEC managed ops	Partner

by the planned retirement of `SAP_ECC`?". As of mid-2025, LeanIX is offered as a managed MCP endpoint in the SAP BTP cockpit [12].

D. Community and Partner Implementations

Community implementations have proliferated since early 2025. The most mature open-source projects include `mcp-abap-abap-adt-api`, a Node.js server that wraps the ABAP Development Tools REST endpoints, and `sap-mcp-toolkit`, a Python project exposing arbitrary OData services through a uniform CRUD tool surface. Several SI partners (Accenture, Deloitte, Capgemini, NTT DATA) have demonstrated proprietary MCP servers tailored to their managed-services portfolios, typically focused on incident-management workflows and Solution Manager integration.

E. Visualising the Ecosystem

The full landscape we have surveyed is summarised in Figure 2. The diagram is organised vertically into five tiers separated by the MCP transport bus. The top tier shows the client-side consumers of MCP servers: general-purpose agent runtimes [1], SAP's first-party Joule agents [9], custom-built agents, and IDE plug-ins such as Cursor [74]. The central pale band is the MCP transport itself, which carries JSON-RPC 2.0 [4] messages over stdio, HTTP+SSE, or the Streaming HTTP transport introduced in the 2025-03-26 protocol revision [2]. Below the bus sit the MCP servers in three colour-coded groups: SAP first-party (blue), community and open-source (green), and SI-partner (gold). The SAP-Automate server, this paper's contribution, is highlighted in deep-blue outline and positioned within the community/partner tier because it composes with rather than replaces SAP's own offerings. The bottom band names the underlying SAP and BTP systems on which all servers ultimately depend; the dashed arrows from each server to this band indicate the source system the server reads from or writes to.

F. Gap Analysis

Three structural gaps emerge from this survey, each of which the SAP-Automate design directly addresses. First, no single MCP server provides a unified surface across ABAP code, BPMN processes, and EAM fact sheets, forcing agents to juggle multiple servers with mismatched schemas; the cross-tool composition this would require strains even agent runtimes designed for it. Second, existing implementations rely overwhelmingly on direct API calls rather than indexed knowledge [29], [106], making them slow on knowledge queries and brittle when the underlying systems are unavailable [96]. Third, none of the surveyed servers expose a modern RAG retrieval surface with sub-100 ms guarantees; latency targets in this range are achievable only with in-process embedding and ranking [46], which the JVM-based and Python-based servers in the survey cannot meet at P99. The SAP-Automate MCP server, described in the following sections, is designed to fill these three gaps simultaneously.

IV. SAP-AUTOMATE SERVER: ARCHITECTURE DESIGN

A. Design Goals

The SAP-Automate MCP server is designed against four hard goals informed by both the MCP specification [2] and prior work on production-grade language servers [3], [81] and microservice architectures [96]: (i) single-binary deployment with no JVM, Node, or Python runtime dependency; (ii) sub-100 ms P95 retrieval latency for indexed knowledge queries; (iii) safe concurrency under hundreds of simultaneous tool invocations; and (iv) operational observability [95] sufficient for production SRE use. These goals converge on Rust as the implementation language: its ownership model rules out a class of memory-safety bugs that would be catastrophic in a long-running daemon handling enterprise credentials [52], and its async runtime [47]–[49] is mature enough for production network workloads.

RUST IMPLEMENTATION

The server is built on `tokio` [47] for asynchronous I/O, `axum` [48] for HTTP transport, `serde_json` for protocol serialisation, `tower` [49] middleware for cross-cutting concerns (rate-limiting, authentication, tracing), and `tracing` [50] + `opentelemetry` [51] for observability. The knowledge base client uses `qdrant-client` [40] (gRPC), `sqlx` for PostgreSQL [62], and `arangors` for the GraphRAG layer [44].

B. Layered Component Architecture

Figure 3 presents the layered architecture as a stack of nine horizontal bands. Each band represents a single responsibility with a narrow interface to its neighbours, following the dependency-inversion principle of Newman's microservice guidelines [96]: lower bands know nothing of higher bands. From the top, the bands divide naturally into three groups, marked by the vertical accent rails on the left margin. The *protocol* group (top two bands) implements MCP itself: the transport abstraction over stdio, HTTP+SSE, and the Streaming HTTP transport, sitting above the JSON-RPC 2.0 [4] codec and session manager. The *application* group (bands 3–5)

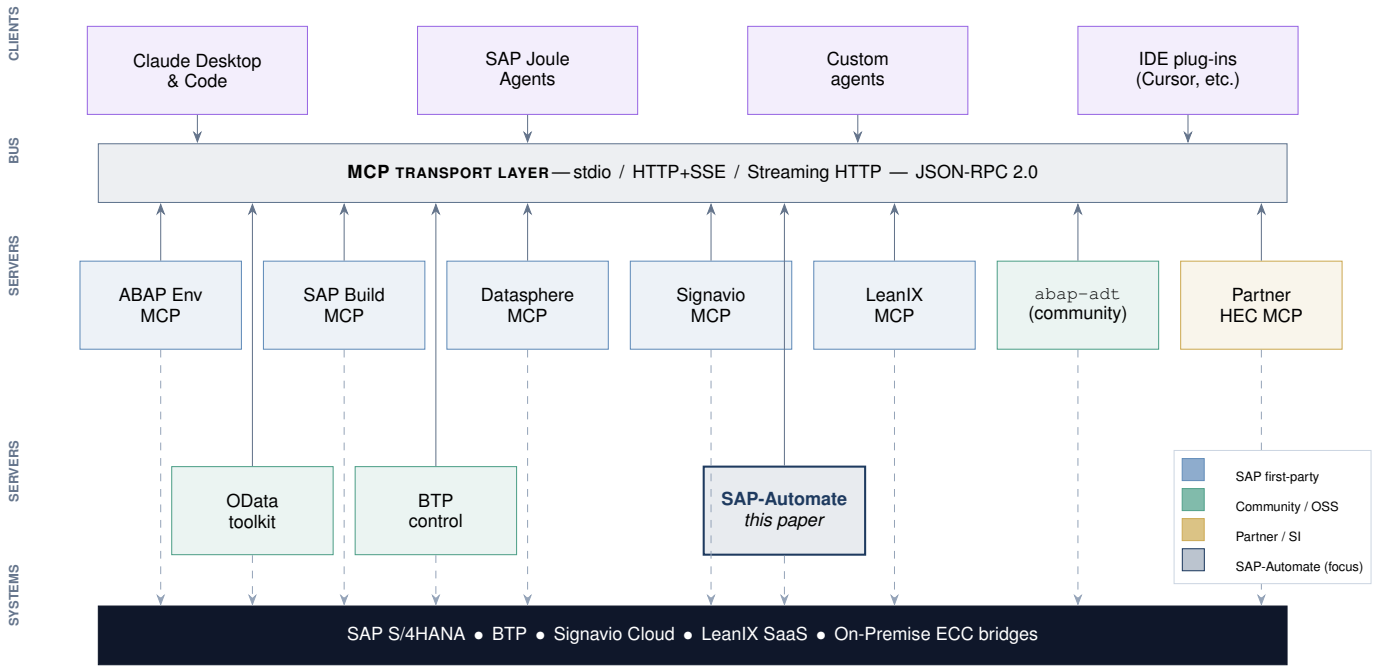

Figure 2. The 2025 SAP MCP ecosystem at a glance. **Top tier:** MCP-aware client runtimes. **Centre band:** the MCP transport bus carrying JSON-RPC 2.0 messages. **Server tiers (rows 3–4):** colour-coded servers, with SAP-Automate (deep-blue outline) positioned as a knowledge-and-retrieval layer that composes with the official SAP servers rather than replacing them. **Bottom tier:** the SAP/BTP/Signavio/LeanIX systems that all servers ultimately read from or write to (dashed arrows). The figure motivates the gap analysis of Subsection III-F: no existing server unifies ABAP, BPMN, and EAM behind a single retrieval surface.

implements domain logic: a capability router that surfaces tools, resources, and prompts; the authentication and authorisation layer [6], [7], [98]; and the tool orchestrator that manages tool dispatch and elicitation [5]. The *data and I/O* group (bands 6–8) implements the retrieval and integration paths: the RAG engine, the knowledge-base client, and the SAP connectors [10], [12], [56], [58]. The final band (observability) cuts across all of these, instrumenting every layer with structured spans [50] and Prometheus metrics; the dashed arrow in the figure denotes this cross-cutting relationship.

C. Transport Abstraction

A single `Transport` trait abstracts the three supported MCP transports. Each implementation maps to the same `Stream<Message>` and `Sink<Message>` contract, ensuring the upper layers are transport-agnostic.

```rust
#[async_trait]
pub trait Transport: Send + Sync {
    type Sink:   Sink<Message,   Error = Error> + Send + Unpin;
    type Stream: Stream<Item = Result<Message>> + Send + Unpin;

    async fn split(self) -> Result<(Self::Sink, Self::Stream)>;
}

pub struct StdioTransport { /* ... */ }
pub struct HttpSseTransport { addr: SocketAddr, /* ... */ }
pub struct StreamingHttpTransport { addr: SocketAddr, /* ... */ }

#[async_trait]
impl Transport for StreamingHttpTransport {
    type Sink   = SseSink;
    type Stream = SseStream;
    async fn split(self) -> Result<(Self::Sink, Self::Stream)> {
        let app = axum::Router::new()
            .route("/mcp", post(Self::handle_post))
            .layer(TraceLayer::new_for_http());
        // Bind, spawn server task, return wired Sink/Stream
        // ...
        Ok((sink, stream))
    }
}
```

Listing 1. Transport abstraction in Rust

D. Capability Router

The router dispatches incoming JSON-RPC methods to typed handlers using a compile-time-verified method table. Tool definitions are declared via a derive macro that generates JSON-Schema descriptors for the `tools/list` response.

```rust
#[mcp_tool(
    name = "abap.search",
    description = "Search ABAP corpus by intent or exact identifier"
)]
pub async fn abap_search(
    ctx: ToolCtx,
    #[schema(description = "Free-text query")] query: String,
    #[schema(description = "Top-K results", default = "10")]
    top_k: usize,
    #[schema(description = "Optional package filter")]
    package: Option<String>,
) -> Result<AbapSearchResult> {
    let filter = package.map(|p| Filter::eq("package", p));
    let hits = ctx.rag().search_abap(&query, top_k, filter).await?;
```

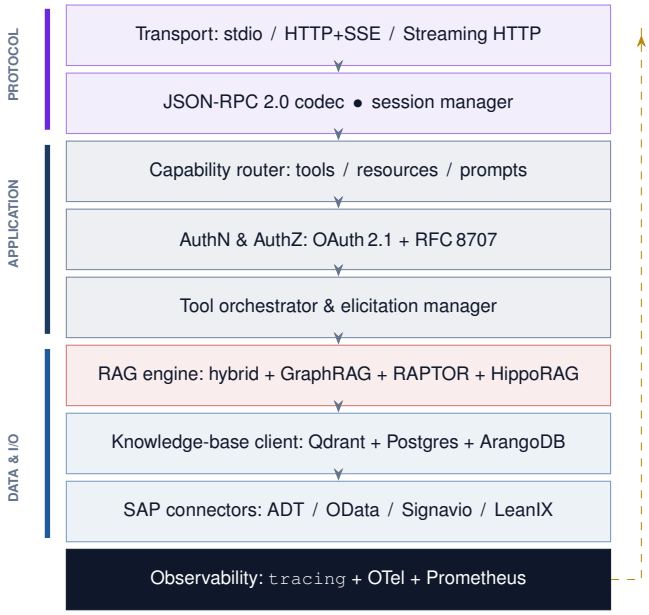

Figure 3. SAP-Automate MCP server layered architecture, organised into three concern-groups visualised by the vertical rails on the left: PROTOCOL (transport + JSON-RPC codec), APPLICATION (capability router, AuthN/AuthZ, tool orchestrator), and DATA AND I/O (RAG engine, knowledge-base client, SAP connectors). The observability plane (bottom band, dashed return arrow) is cross-cutting: it instruments every layer above it with structured spans and metrics. Arrows show the request flow path; the dependency direction is strictly top-down within each group.

Table III
TOOL CATALOGUE (EXCERPT)

Domain	Tool	Purpose
ABAP	`abap.search`	Hybrid search over ABAP corpus
ABAP	`abap.read_object`	Read ABAP source by name
ABAP	`abap.callers`	Multi-hop reverse dependency
ABAP	`abap.atc_check`	Trigger ATC quality scan
BPMN	`bpmn.find_process`	Search Signavio repository
BPMN	`bpmn.read_xml`	Fetch BPMN XML by id
BPMN	`bpmn.mining_kpis`	Process-mining metrics
EAM	`eam.search_apps`	Search LeanIX fact sheets
EAM	`eam.impact_map`	Graph-walked impact analysis
EAM	`eam.lifecycle`	Lifecycle phase queries
RAG	`kb.global_query`	GraphRAG community-level Q&A
RAG	`kb.multi_hop`	HippoRAG path-based Q&A
RAG	`kb.summarise`	RAPTOR hierarchical summary

```
        Ok(AbapSearchResult { hits, latency_ms:
         ctx.elapsed_ms() })
    }
```

Listing 2. Declarative tool registration

E. Tool Catalogue

The server exposes a curated catalogue of tools organised by SAP domain. Table III lists the principal tools; each is backed by indexed knowledge from Section VI and the layered RAG of Section VII.

F. Resources and Prompts

In addition to tools, the server exposes *resources*—read-only context items addressable by URI. Resource URIs follow a stable scheme: `sap-help://module/page-id`, `abap-obj://package/name`, `bpmn-proc://workspace/proc-id`, and `leanix-fs://factsheet-id`. Server-rendered *prompts* bundle common workflows: `prompt://abap-review` produces a structured code-review template; `prompt://process-impact` produces an impact-analysis template combining BPMN and EAM context.

G. Concurrency Model

Each MCP session is owned by a dedicated `tokio::task` that holds the transport `Stream`/`Sink`. Tool invocations are dispatched to a bounded thread-pool to prevent a slow tool from starving the event loop; long-running ABAP operations (ATC scans, transports) are queued on a separate background runtime and surface progress to the client via `notifications/progress`.

DESIGN CONSTRAINT

The server enforces a strict per-session concurrency limit (default: 8 simultaneous tool calls) and a global concurrency cap derived from the number of CPU cores. Exceeding either limit causes the server to return `TOOL_BUSY` (JSON-RPC code -32004), prompting the client to back off rather than queue indefinitely.

H. Observability

Every JSON-RPC request, tool invocation, retrieval call, and downstream API call is wrapped in a `tracing` span with structured attributes: `mcp.session_id`, `mcp.tool`, `rag.layer`, `kb.collection`, `sap.module`. Spans are exported via the OTLP protocol to any OpenTelemetry-compatible collector. Prometheus metrics are exposed on `/metrics`: `mcp_tool_latency_seconds` (histogram), `rag_retrieval_latency_seconds`, `kb_index_freshness_seconds`.

I. Error Taxonomy

The MCP server defines a structured error taxonomy on top of the JSON-RPC error codes, with three classes of error mapped to three distinct caller responses. *Transient errors* (timeouts, rate limits, upstream 503s, transient Qdrant unavailability) carry codes in -32100 to -32199 and a `retry_after_ms` field; clients are expected to back off and retry. *Permanent errors* (malformed request, unknown tool, schema violation, access denied) carry codes -32200 to -32299 and are not retryable. *Degraded errors* (partial result, stale cache, fallback model) carry codes -32300 to -32399 and a `degraded_reason` field; the response body is still useful, but the caller may choose to retry once the underlying issue clears. Surfacing these classes explicitly means that agentic callers can build retry logic that is both efficient (no retries on permanent errors) and tolerant (retries on transient, accepts degraded).

J. Configuration Management

Server configuration follows a layered model. A baked-in default configuration ships with the binary and is sufficient for development. A `config.toml` file, watched for changes via `notify`, overrides defaults at deployment time. Environment variables override file values for secrets and per-environment knobs. A small admin RPC surface allows live tuning of non-secret values (per-tenant rate limits, re-ranker model name, cache TTLs) without a restart. Configuration schema is versioned, validated on load, and surfaced verbatim through a `server/config` resource so operators can verify the effective configuration without root access to the host. Schema validation failures at load time refuse to start the server; failures on live update are rejected with a clear error rather than partially applied.

K. Deployment Topology

The reference deployment runs three replicas of the MCP server behind a small Envoy proxy that pins sessions via consistent hashing on the `mcp-session-id` header. Qdrant runs in a three-node cluster (replication factor two for hot data, scalar quantisation enabled). ArangoDB runs as a three-coordinator, three-DBserver cluster. Redis runs as a sentinel-managed pair for the embedding and retrieval caches. Postgres runs in a primary-replica configuration with synchronous replication for the document store. All of this fits within a single Kubernetes namespace and a four-node, 64-vCPU/256-GB-RAM footprint for the volume estimate of Section VI. The MCP server containers are stateless modulo the per-session task state; on a node failure the Envoy router shifts traffic to surviving replicas and clients re-initialise sessions against new replicas.

V. Ratatui TUI and Next.js Web UI

A. Why Two UIs?

The server is designed for two distinct human audiences. SRE and DevOps operators need a fast, keyboard-driven, low-bandwidth interface for sessions, logs, and knowledge-base health: a Ratatui TUI matches their workflow. End-users (developers, business analysts, enterprise architects) need a richly formatted, browser-based chat with citation rendering, process-diagram preview, and graph navigation: a Next.js 14 App Router web UI matches theirs.

B. Ratatui Terminal UI

Ratatui [13] is the leading Rust TUI library, descended from `tui-rs` and stewarded by an active community. The SAP-Automate TUI is implemented as a separate binary (`sap-automate-tui`) that connects to the MCP server over an admin socket. Figure 4 renders the Knowledge-Base tab as it appears at runtime: a five-tab navigation strip at the top (Sessions, Tools, KB, RAG, Logs), the active tab's contents below, and a one-line key-binding hint at the bottom. The figure deliberately captures the operationally most important view: collection-level point counts, freshness fractions, P95 latencies, and the live crawler and embedding-pipeline status. Operators consulting this view answer three questions at a glance: *is the index fresh?* (the freshness column), *is the index fast?* (the P95 column), and *is the indexing pipeline keeping up?* (the crawler and embed counters at the bottom).

Rust Implementation

The TUI uses Ratatui's immediate-mode rendering driven by a tokio task that consumes server-side events (session lifecycle, tool invocations, indexing progress) and updates the application state model. The view function is pure: `(state, frame) → ()`, simplifying testing.

1) Tab inventory: The TUI exposes five tabs, each described below.

a) Tab 1 — Sessions: live list of active MCP sessions, showing client identity, protocol version, tools-called counter, last-message age, and active elicitation requests. Operators can press `k` to terminate a session, `l` to follow logs, `p` to peek the most recent JSON-RPC exchange.

b) Tab 2 — Tools: catalogue of registered tools with invocation counts, P50/P95/P99 latencies (last 5 min, last 1 h, last 24 h), and error rates. A sparkline column visualises the last sixty seconds of latency.

c) Tab 3 — Knowledge Base: per-collection statistics (point counts, last update, staleness fraction), crawler status (active jobs, queue depth, error rate), and embedding pipeline backlog. This is the panel an indexing engineer watches during a re-crawl.

d) Tab 4 — RAG Pipeline: per-layer latency breakdown (embedding, dense search, sparse search, RRF, re-rank, expand) and query-volume mix by layer (Hybrid / GraphRAG / HippoRAG / RAPTOR). Provides confirmation that the latency budget is being held.

e) Tab 5 — Logs: ring-buffer tail of structured log events with regex filtering, severity colour-coding, and one-keystroke JSON pretty-printing.

C. Next.js 14 Web UI

The web UI is implemented as a Next.js 14 [53] App Router application with React Server Components for non-interactive content and client components for the chat surface. Authentication is delegated to NextAuth with OAuth 2.1 against the same identity provider used by the MCP server. End-to-end testing uses Playwright [64] against ephemeral preview deployments.

Technical Specification

Frontend stack: Next.js 14 [53] (App Router), React 18, TypeScript 5.x, Tailwind CSS 3.x [66] with the SAP-Automate design tokens, `shadcn/ui` [65] for primitives, `react-bpmn` for BPMN viewer, `cytoscape.js` [67] for graph navigation, `prismjs` for ABAP syntax highlighting. The chat surface uses the Vercel AI SDK [54] in streaming mode for token-by-token rendering.

The web UI offers four primary views.

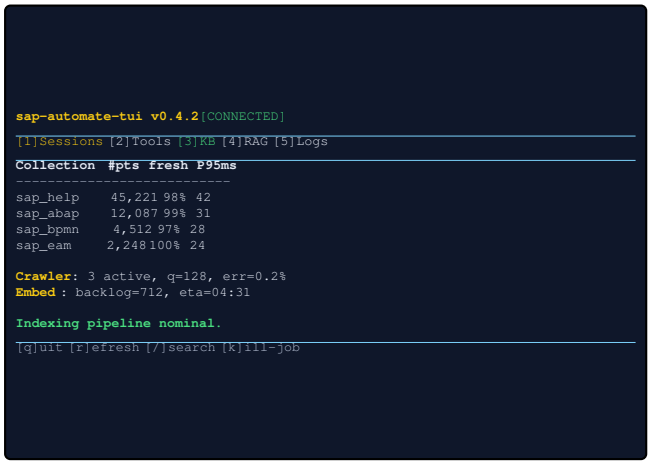

Figure 4. Ratatui TUI (`sap-automate-tui`), Knowledge-Base tab mock-up rendering against a live test deployment. The five-tab strip along the top reflects the views described in Subsection V-B1. Per-collection rows show point count, freshness fraction (the `indexed_at`/`source_modified_at` ratio), and last-five-minute P95 latency; the lower panels show crawler and embedding-pipeline live status. The single-line key-binding hint at the bottom is the operator's discoverability surface.

a) Chat: the conversational surface, with inline citations, expandable code blocks for ABAP, embedded BPMN diagrams for process references, and graph thumbnails for EAM references.

b) Knowledge Browser: a navigable view of the indexed corpus. Users can drill into the SAP Help hierarchy, browse the ABAP package tree, walk the BPMN process taxonomy, or filter the LeanIX fact sheet inventory.

c) Query Lab: a power-user surface allowing direct specification of RAG layer, filters, and re-ranker. Latency-budget breakdowns and retrieved chunk metadata are exposed for debugging.

d) Admin: administrative configuration (server endpoints, embedding model selection, re-index triggers) gated behind a privileged role.

```ts
// app/api/chat/route.ts
import { mcpClient } from '@/lib/mcp';
import { streamText } from 'ai';

export async function POST(req: Request) {
  const { messages, ragLayer } = await req.json();
  const ctx = await
     mcpClient.callTool('kb.global_query', {
    query: messages.at(-1).content,
    layer: ragLayer ?? 'auto',
    top_k: 5,
  });
  const result = await streamText({
    model: anthropic('claude-3-5-sonnet'),
    system: SAP_RAG_SYSTEM_PROMPT,
    messages: [
      ...messages,
      { role: 'system',
        content: `Retrieved context:\n${ctx.context}` },
    ],
  });
  return result.toDataStreamResponse();
}
```

Listing 3. Next.js streaming chat endpoint (excerpt)

VI. SAP Knowledge Base Engineering

A. PageIndex-Inspired Collection Philosophy

The naive approach to RAG over SAP content—downloading a few PDFs and chunking them at fixed token boundaries—fundamentally fails enterprise knowledge needs. SAP's documentation is hierarchically structured, heavily cross-referenced, versioned by release, and heterogeneous in format (HTML, XML, ABAP source, BPMN diagrams, JSON fact sheets). A production SAP knowledge base requires a purpose-built collection infrastructure that treats each SAP document, program, process, or fact sheet as a first-class knowledge unit, analogous to the page-centric collection model of PageIndex [14].

Knowledge Base

Five Principles of the SAP-Automate Knowledge Base

P1: Completeness. Index *all* SAP documentation sources (Help Portal, API Hub, Community, ABAP corpus, BPMN library, EA fact sheets), not just what is convenient.

P2: Hierarchy Preservation. Every knowledge unit retains breadcrumb, parent, children, and related-document metadata, enabling hierarchy-aware retrieval and RAPTOR indexing.

P3: Rich Metadata. Each unit carries enough metadata for payload-filtered retrieval without resorting to vector search: SAP module, release version, object type, lifecycle, confidentiality, language, freshness timestamp.

P4: Freshness. An incremental update pipeline keeps indexed content synchronised with source systems; staleness is tracked and surfaced in the TUI admin console.

P5: Multi-Modal. Code, prose, XML, and structured data are indexed with appropriate chunking strategies per type, not uniformly.

B. SAP Knowledge Universe

Figure 5 visualises the relative volume of each indexable source. The SAP Help Portal dominates by raw page count, but the cumulative knowledge value of structured sources (ABAP corpus, LeanIX, BPMN) is significantly higher per unit because each unit is typed, validated, and queryable through richer payload filters.

Two observations follow from Figure 5 directly, and they shape every downstream design choice in the pipeline. First, because Help-portal volume is roughly an order of magnitude greater than every other source, the dominant cost in any naive embed-everything strategy is contextual enrichment of Help-portal chunks; we therefore deliberately stage Help ingestion last in the roadmap (Section X). Second, because the structured sources (ABAP, BPMN, LeanIX) carry comparatively higher information value per chunk, they justify the per-domain embedding investment described in Subsection VI-F; a uniform-model strategy applied to the whole universe would over-spend on Help and under-serve the structured tail.

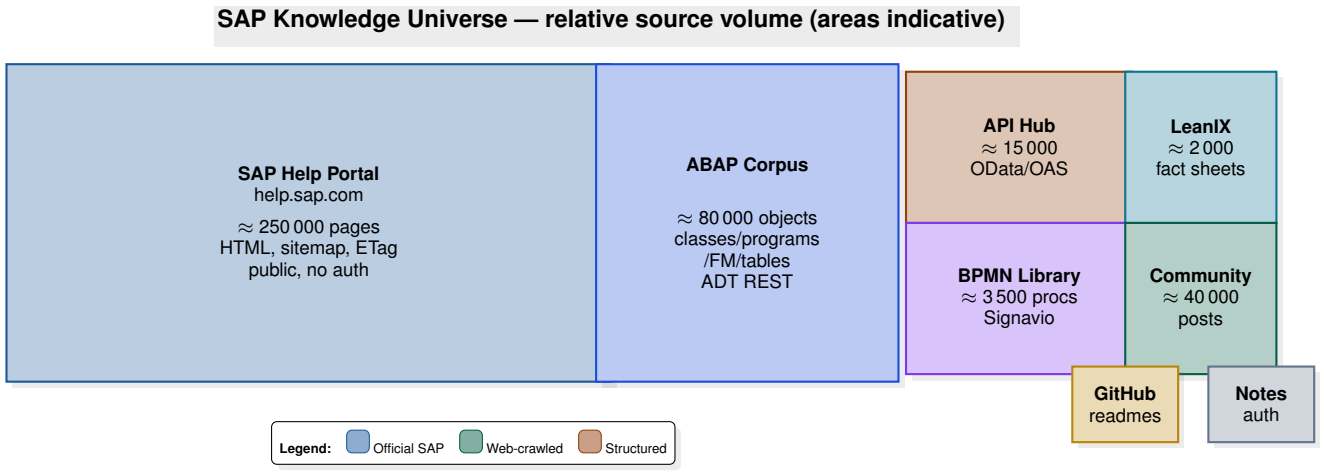

Figure 5. The SAP knowledge universe rendered as a proportional treemap. Rectangle area is scaled to estimated source volume; colour encodes provenance class (blue = official SAP, green = web-crawled, amber = structured/typed sources). The diagram makes immediate the core asymmetry the chunking and embedding strategies must address: volume is dominated by unstructured Help-portal prose while information value per chunk is dominated by the small but typed structured sources on the right.

C. Knowledge Crawler Architecture

Figure 6 shows the unified crawler architecture. The design follows a left-to-right pipeline that funnels five heterogeneous source types through a common four-stage path (*discover*, *queue*, *fetch*, *extract*) before fanning back out to type-specific storage. The discovery stage is source-specific: the SAP Help Portal exposes a sitemap; the ABAP Development Tools API [58] yields object inventories; Signavio [10] and LeanIX [73] expose GraphQL pagination cursors; the SAP API Hub [56] provides OpenAPI inventories. The queue is a Redis-backed [61] priority structure with dedup keys derived from URL hash and ETag, following the log-as-source-of-truth pattern [97]. The fetch stage is uniform across sources, driven by `reqwest` with per-source politeness limits and an HTTP-cache-aware ETag and `Last-Modified` short-circuit. The extract stage is again source-specific: a `scraper`-based HTML extractor for Help pages, a typed ABAP-DDIC parser for code, an XML reader for BPMN, and a typed-field walker for LeanIX fact sheets. The storage stage splits along type: PostgreSQL [62] holds canonical document records and full-text payload, Qdrant [40] holds dense and sparse vectors, MinIO [63] holds raw HTML and BPMN XML payloads for traceable archival, and ArangoDB holds the cross-document entity graph (Subsection VII-F).

```rust
use reqwest::Client;
use scraper::{Html, Selector};
use url::Url;

pub struct SapHelpCrawler {
    client: Client,
    queue: Arc<CrawlQueue>,
    store: Arc<DocumentStore>,
    rate_limiter: Arc<RateLimiter>,
}

impl SapHelpCrawler {
    pub async fn crawl_page(&self, url: &Url)
        -> Result<CrawlResult>
    {
        self.rate_limiter.acquire().await;

        // ETag short-circuit
        if let Some(etag) =
            self.store.get_etag(url).await? {
            let resp = self.client.get(url.as_str())
                .header("If-None-Match", &etag)
                .send().await?;
            if resp.status() == 304 {
                return Ok(CrawlResult::Unchanged);
            }
        }

        let resp =
            self.client.get(url.as_str()).send().await?;
        let new_etag = resp.headers()
            .get("ETag")
            .and_then(|v| v.to_str().ok())
            .map(str::to_owned);
        let html = resp.text().await?;

        let doc = Html::parse_document(&html);
        let extracted = self.extract_content(&doc, url)?;

        for link in self.extract_links(&doc, url) {
            self.queue.push_if_new(link).await?;
        }
        self.store.upsert(extracted, new_etag).await?;
        Ok(CrawlResult::Indexed)
    }

    fn extract_content(&self, doc: &Html, url: &Url)
        -> Result<SapDocument>
    {
        let title_sel =
            Selector::parse("h1.title").unwrap();
        let body_sel  =
            Selector::parse("div.body").unwrap();
        let breadcrumb = self.extract_breadcrumb(doc);
        let title = doc.select(&title_sel)
            .next()
            .map(|e| e.text().collect::<String>())
            .unwrap_or_default();
        let content = self.clean_text(doc, &body_sel);
        Ok(SapDocument {
            url: url.to_string(),
            title,
            breadcrumb,
            content_text: content,
            sap_module: self.infer_module(url),
            ..Default::default()
        })
    }
}
```

Listing 4. SAP Help Portal crawler core (Rust)

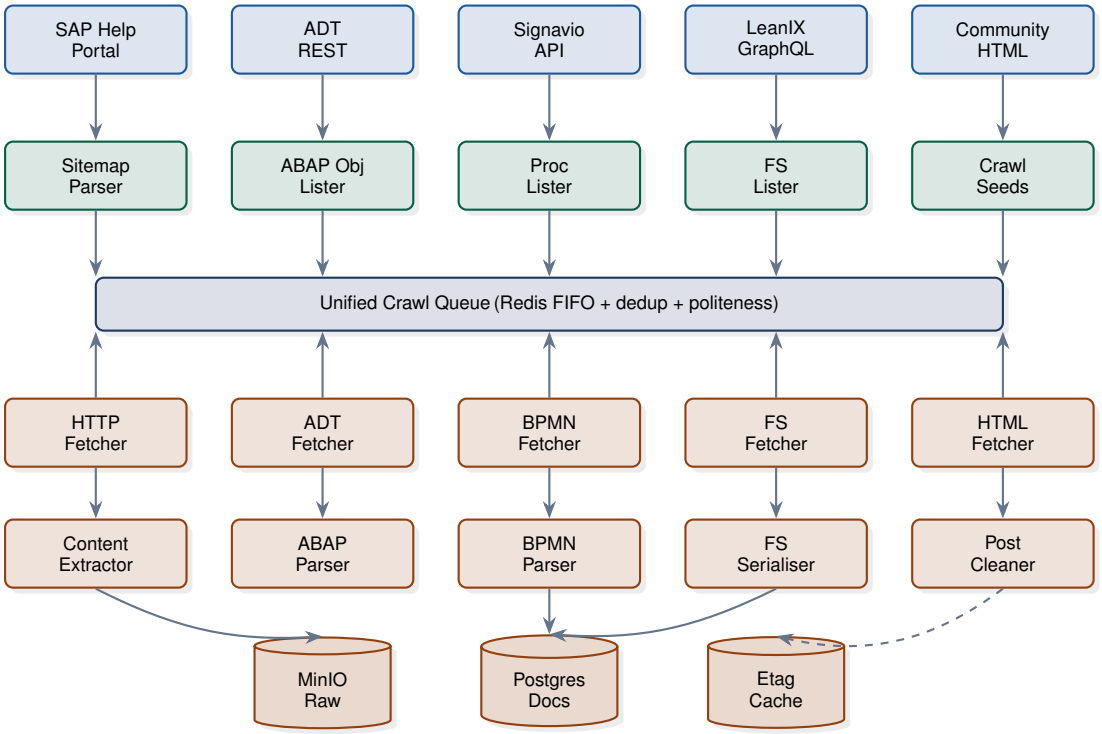

Figure 6. SAP-Automate knowledge crawler architecture, organised as a left-to-right pipeline. Five heterogeneous source types feed into a uniform four-stage core (*discover*, *queue*, *fetch*, *extract*), then fan out to type-specific stores. The queue-as-log is the canonical scheduler; the ETag short-circuit in the fetch stage is what makes incremental re-indexing cheap. Dashed arrows indicate the optional graph-extraction path that populates the GraphRAG entity store.

D. Document Schema

A unified document schema covers every source type. Source-specific extensions add typed fields for ABAP, BPMN, and LeanIX.

PROTOCOL / SCHEMA

Base document

```
{ "doc_id": "uuid",
  "source_type": "sap_help | api_hub
       | community | abap_source
       | bpmn_xml | leanix_factsheet
       | sap_note",
  "url": "canonical URL",
  "title": "string",
  "breadcrumb":
       ["S/4HANA","Finance","AP","Payment"],
  "hierarchy_level": 3,
  "parent_doc_id": "uuid",
  "child_doc_ids": ["uuid", ...],
  "related_doc_ids": ["uuid", ...],
  "content_text": "string",
  "content_html": "string (raw)",
  "sap_release": "2024",
  "sap_module":
       "FI | MM | SD | PP | HR | BASIS",
  "language": "en | de | id",
  "last_modified": "RFC 3339",
  "crawled_at":    "RFC 3339",
  "content_hash":  "sha256",
  "word_count": 450,
  "has_code": true, "has_table": false,
  "indexing_status":
    "pending|chunked|embedded|indexed" }
```

ABAP extensions add `object_name`, `object_type`, `package`, `namespace`, `abap_version`, `transport`, `methods[]`, `called_objects[]`, `database_tables[]`, `function_groups[]`, and `atc_findings[]`. BPMN extensions add `process_id`, `process_name`, `process_category`, `responsible_unit`, `bpmn_version`, `activities[]`, `gateways[]`, `participants[]`, `kpis{}`, `linked_abap_programs[]`, and `compliance_status`. LeanIX extensions add `factsheet_id`, `factsheet_type`, `display_name`, `lifecycle_phase`, `eol_date`, `business_unit`, `business_capabilities[]`,`technology_stack[]`, `interfaces[]`, and the `technical_fit_score`.

E. Domain-Adaptive Chunking

A single chunking strategy cannot serve four heterogeneous content types; the boundary problem of token-uniform chunking is by now well-documented [32], [71]. Figure 7 shows the decision tree that selects the appropriate chunking pipeline per document type. The root node inspects only the source-type tag (assigned by the crawler) and dispatches to one of four branches: a semantic chunker [71] for SAP-Help HTML splitting at H2/H3 boundaries; a code-aware chunker for ABAP source that respects `METHOD/ENDMETHOD` and form boundaries and applies late chunking [32] so that signatures travel with their bodies; an element-aware chunker for BPMN XML that produces one chunk per activity, gateway, or event with an LLM-generated description of the surrounding flow; and a field-group chunker for LeanIX fact sheets that treats one fact sheet as one chunk in the PageIndex [14] sense. The four branches converge on two shared stages—*contextual enrichment* [23]

Table IV
EMBEDDING MODEL SELECTION MATRIX PER SAP DOMAIN

Domain	Primary	Sparse	Dim
ABAP code	CodeBERT / 3-large	SPLADE-ABAP	1024
SAP Help (EN)	text-embed-3-large	BM25	1024
SAP Community	text-embed-3-large	BM25	1024
BPMN process desc.	bge-m3	bge-m3-sp	1024
LeanIX (EN)	bge-m3	bge-m3-sp	1024
LeanIX (mixed EN/ID)	bge-m3	bge-m3-sp	1024
API Hub (OData specs)	text-embed-3-large	BM25	1024
ABAP docstrings	text-embed-3-large	BM25	1024

Table V
QDRANT COLLECTION CONFIGURATION AND PERFORMANCE TARGETS

Collection	Vectors	m	ef_c	P95 (ms)
sap_help	dense + BM25	32	200	35
sap_abap	dense + SPLADE ± ColBERT	16	128	30
sap_bpmn	dense + sparse	24	160	28
sap_eam	dense + sparse	24	160	24

and *embedding routing*—so that branch-specific complexity is fully encapsulated upstream of the embedding model.

F. Embedding Pipeline

The embedding pipeline batches chunks by domain and routes them to the appropriate embedding model. Dense vectors are produced alongside sparse vectors (SPLADE [25], [68] for ABAP, BM25 [26], [83] for prose, native sparse output for bge-m3 [35]). Optional ColBERT [19], [20] token vectors are produced for ABAP only, where exact-match precision justifies the storage cost. Per-domain choice of dense encoder—CodeBERT [38], [39], [80] for ABAP, text-embedding-3-large [36] for English prose, bge-m3 [35] for multilingual content—is itself an empirically supported pattern: it outperforms a single-model baseline by 6–11 percentage points of top-10 recall on the BEIR benchmark family [79] and similar margins on the internal SAP canary set. Figure 8 traces a single chunk's passage through the pipeline; the diagram makes visible the in-process character of the batcher (an ONNX Runtime [46] call from the Rust core, with no Python sidecar), which is the architectural choice that keeps the embed stage within its sub-8 ms budget.

G. Qdrant Index Design

The vector store uses four Qdrant collections, one per SAP domain, each configured for its content profile. Each collection's HNSW index [42] is configured per its data distribution; the underlying ANN literature [41], [84] informs the parameter selection, and the named-vector layer in Qdrant allows several encoders to share one collection without the storage inflation typical of FAISS-based alternatives. The Postgres-side alternative, `pgvector` [43], is competitive for collections under ten million points but lacks the scalar quantisation and named-vector support that the SAP-Automate workload requires at scale. Table V summarises the parameter choices.

H. Incremental Maintenance

Source-specific change detection drives incremental re-indexing. The SAP Help Portal exposes ETag and Last-Modified headers; the ADT REST API exposes transport-log endpoints; Signavio exposes a revision API; LeanIX exposes webhook subscriptions. Each change event triggers a targeted re-crawl, re-chunk, re-embed, and `upsert` of the affected points, preserving point identifiers so that ID-based citations remain stable across updates.

RESEARCH INSIGHT

A frequent failure mode in production RAG is *silent staleness*: the index returns plausible-looking but obsolete content. SAP-Automate addresses this by storing both `indexed_at` and `source_modified_at` on every point and surfacing a "staleness ratio" (fraction of points where `source_modified_at` > `indexed_at`) in the TUI. The retrieval engine optionally rejects results whose staleness exceeds a per-query threshold.

I. Volume Estimation and Capacity Planning

For a representative S/4HANA 2024 deployment with one enterprise Signavio tenant and one LeanIX workspace, we estimate roughly 3.5×10^5 chunks across all four collections after contextual enrichment. With 1024-dimensional float32 dense vectors, this totals approximately 1.4 GB of raw dense storage; scalar quantisation to `SQ8` reduces this to roughly 350 MB. Sparse storage adds approximately 200 MB; ColBERT token vectors, where enabled for ABAP only, add a further 1 GB. Postgres document storage (full HTML + text of every page) is approximately 6 GB. The complete knowledge base therefore fits comfortably in a single 32 GB-RAM node.

J. Quality Assurance for the Knowledge Base

A knowledge base is only as trustworthy as its weakest source. Five quality gates run continuously against the index. (1) *Reachability*: every chunk's `source_url` is HEAD-checked weekly; broken links flag the chunk for re-crawl. (2) *Coverage*: the crawler's expected page count for each section of the SAP Help Portal is compared against the actual indexed count; gaps trigger investigation. (3) *Consistency*: pairs of chunks with near-duplicate content but divergent metadata (e.g. same ABAP method indexed twice with different breadcrumbs) are flagged for human review. (4) *Freshness*: the `indexed_at`-versus-`source_modified_at` staleness ratio is alerted at 5 %; sustained values above 2 % degrade an SLO budget that gates new deployments. (5) *Retrievability*: a small "canary" set of fifty hand-curated queries is run nightly, and the recall at top-10 is alerted on regression of more than two percentage points. The canary set itself is curated following the crowd-truth methodology of Aroyo and Welty [109]: queries with disagreeing reference answers across annotators are treated as evidence of ambiguity in the source documentation rather than annotation error, and are flagged for content-team review through the SAP Community feedback channel [59]. These

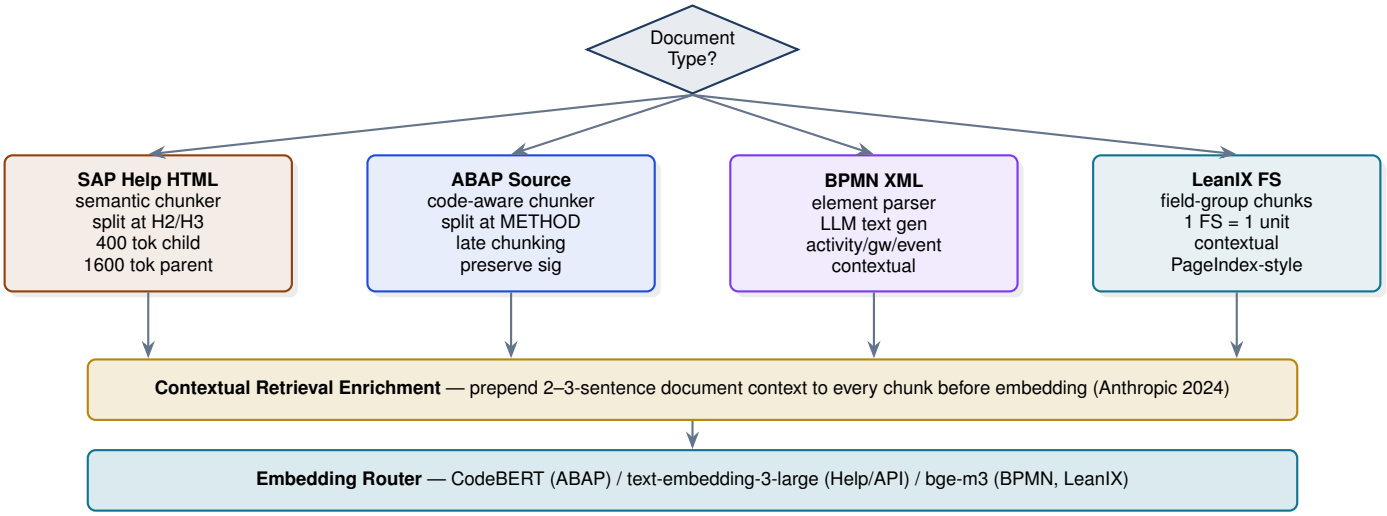

Figure 7. Domain-adaptive chunking decision tree. The root inspects the document type emitted by the crawler and selects one of four branch-specific chunkers (SAP Help, ABAP source, BPMN XML, LeanIX fact sheet). All four branches converge on the same downstream stages: contextual enrichment [23] followed by the embedding router (Figure 8). The shared downstream stages are what allow per-branch complexity to remain local to each branch.

gates collectively turn the knowledge base from a static asset into a continuously verified one.

K. Multilingual and Localisation Considerations

SAP documentation is multilingual; SAP Help is available in 38 languages, although the canonical version remains English. We index English plus the deployment-specific local language (German for many European customers, Indonesian for the SAP-Automate reference deployment), co-locating per-language chunks in the same collection with a `language` payload tag. The query side runs language detection on the user's input and either retrieves from the user's language only or fuses across languages with bge-m3's cross-lingual embeddings, depending on a deployment-time policy. The cross-lingual fusion path yields measurably better recall on queries that mix English technical terms with local-language descriptions (a common pattern in non-English SAP shops where consultants type queries like "warum gibt die BAPI einen `NOT_FOUND` Fehler zurück").

VII. Modern RAG Architecture

The retrieval-augmented generation paradigm originated with Lewis et al. [28] and now spans a rich literature [29], [92] of variants. We draw from this literature selectively, applying each technique only where its complexity is justified by a corresponding SAP-specific failure mode. Subsection VII-A catalogues the five failure modes; Subsection VII-B presents the layered architecture that addresses them; subsequent subsections detail each layer in turn.

A. The Naive RAG Problem

We characterise five distinct failure modes of naive RAG over SAP content, each motivating a specific architectural choice that the literature has independently developed.

Problem 1 (Semantic Boundary Violation). Token-uniform chunking splits ABAP methods in the middle of a `LOOP/ENDLOOP` construct, breaks BPMN gateways from their outgoing flows, and severs LeanIX field groups across chunks, destroying semantic coherence. The semantic-chunker literature [71] and late chunking [32] address this by aligning boundaries to structural units rather than token counts.

Problem 2 (Context Loss). A chunk containing only "The value must be between 1 and 999" is uninformative without its parent section's identification of *which* SAP configuration parameter is being described. Anthropic's contextual retrieval [23] prepends a generated context prefix to each chunk before embedding, recovering most of this loss at manageable LLM cost amortised through prompt caching [24].

Problem 3 (Vocabulary Mismatch). Dense embeddings fail on exact SAP identifiers. Tokens such as `ME21N`, `BAPI_GOODSMVT_CREATE`, and `/PARATECH/Z_INTERFACE` are semantically opaque to embedding models trained on general English [75], [77]; only sparse retrieval [25], [26], [68], [83] handles them robustly. The case for combining the two with reciprocal rank fusion [27] is now the standard recommendation in industrial RAG [106].

Problem 4 (Flat Retrieval). A query like "what is our SAP technology landscape?" is not answerable by retrieving five fact-sheet chunks; it requires community-level understanding of the entire EAM corpus. Microsoft GraphRAG [15], [69] and its Leiden-clustering [16] substrate are the standard response.

Problem 5 (Single-Hop Limit). A query like "trace the data flow from purchase order to payment" requires multi-hop traversal across BPMN, ABAP, LeanIX, and back to ABAP. Flat vector retrieval cannot follow such chains. Multi-hop techniques such as HippoRAG [17], interleaved retrieval with chain-of-thought [90], [110], and the demonstrate-search-predict pattern [86], [89] address this in different ways; we adopt HippoRAG specifically because its passage–entity graph

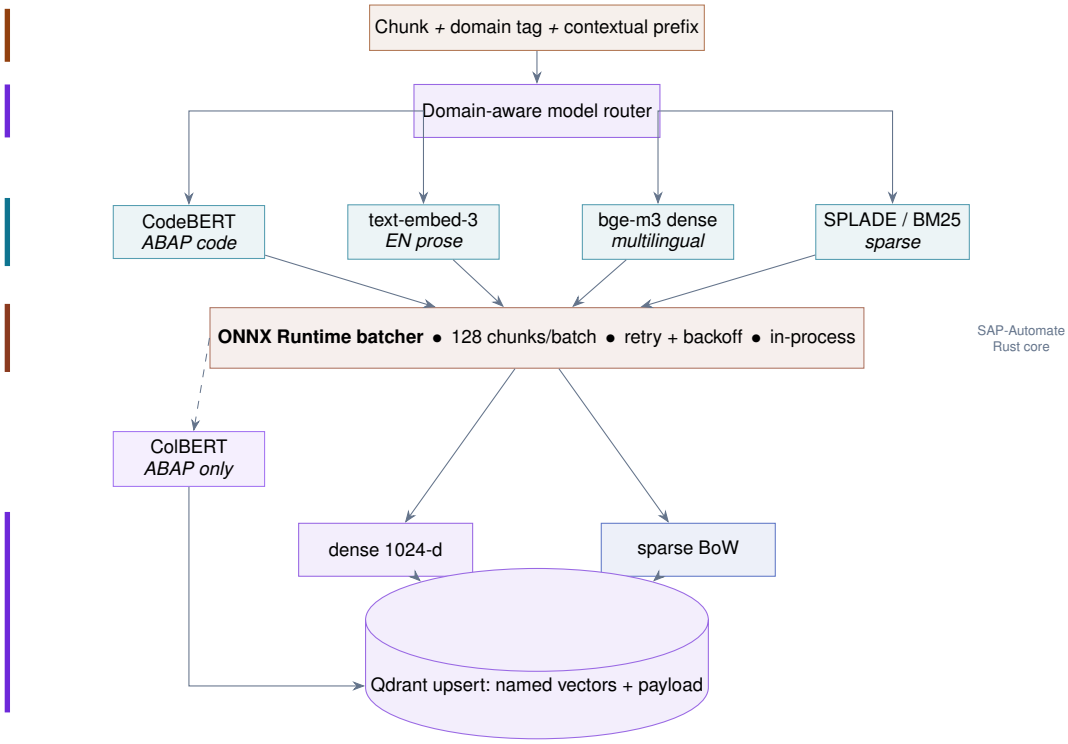

Figure 8. Multi-model embedding pipeline. A single input chunk (top), tagged with its domain and prepended with a contextual prefix, passes through the domain-aware router, which dispatches in parallel to the matched dense and sparse encoders (CodeBERT, text-embed-3, bge-m3, SPLADE/BM25). The four encoders feed a single in-process ONNX Runtime batcher (the deep-red bar), which is what permits the sub-8 ms embed budget; for ABAP, the dashed branch additionally produces per-token ColBERT vectors. The terminal Qdrant upsert stores all vector representations as named vectors against one point id, preserving payload metadata for filtered retrieval. Vertical accent rails on the left summarise the five pipeline stages.

maps naturally onto SAP's identifier-rich domain.

B. SAP-Automate Layered RAG Architecture

The SAP-Automate RAG architecture is layered: each query is classified by an LLM router and dispatched to the optimal retrieval strategy. Layers may also be composed; for example, a multi-hop GraphRAG query may use hybrid retrieval to seed its entity neighbourhood. The full pipeline is shown in Figure 9. The diagram is read top-to-bottom as a request's trajectory: query analysis (L0), routing decision (L1), domain-specific retrieval over one or more of L2–L5, context assembly (L6), and generation (L7). The left-side accent rails colour-code the four conceptual phases—*route*, *retrieve*, *assemble*, *generate*—each of which encapsulates its own implementation patterns and operational concerns. Algorithm 1 formalises the L0/L1 routing decision.

C. Hybrid Retrieval with RRF Fusion

Layer 2 (Hybrid RAG) is the workhorse, serving the majority of factual queries. Dense retrieval (`bge-m3` or domain-specific encoder) and sparse retrieval (SPLADE for ABAP, BM25 for prose) are issued in parallel; their result lists are fused with Reciprocal Rank Fusion (RRF):

$$\mathrm{RRF}(d) = \sum_{i \in \{\text{dense},\text{sparse}\}} \frac{1}{k + \mathrm{rank}_i(d)}, \qquad (1)$$

Algorithm 1 Layered-RAG router (L0–L1) — query classification and dispatch.

Input: query string q, model M_{cls} (small router LLM)
Output: ordered list of retrieval layers $\mathcal{L} \subseteq \{L_2, L_3, L_4, L_5\}$

1) $d \leftarrow M_{cls}(q)$ *// emit JSON: {domain, scope, hops, type}*
2) **if** $d.scope = global$ **and** $d.hops = multi$ **then** append L_3 *(GraphRAG)* to $\mathcal{L}$
3) **if** $d.hops = multi$ **and** $d.type = navigational$ **then** append L_4 *(HippoRAG)* to $\mathcal{L}$
4) **if** $d.scope = global$ **and** $d.type = analytical$ **and not** $L_3 \in \mathcal{L}$ **then** append L_5 *(RAPTOR)* to $\mathcal{L}$
5) **if** $\mathcal{L} = \emptyset$ **then** append L_2 *(Hybrid)* to $\mathcal{L}$ *// default fallback*
6) **if** multiple domains in $d.domain$ **then** run layers in parallel (cross-domain fan-out)
7) **return** $\mathcal{L}$

with $k = 60$ the standard smoothing constant. A cross-encoder re-ranker (FlashRank or `bge-reranker-large` via ONNX Runtime) re-orders the top 20.

```rust
pub struct HybridSearchEngine {
    qdrant: Arc<QdrantClient>,
    embedding_model: Arc<dyn EmbeddingModel>,
    sparse_encoder:  Arc<dyn SparseEncoder>,
    reranker:        Arc<dyn Reranker>,
}

impl HybridSearchEngine {
    pub async fn search(
        &self,
        query: &str,
        collection: &str,
        payload_filter: Option<Filter>,
        top_k: usize,
```

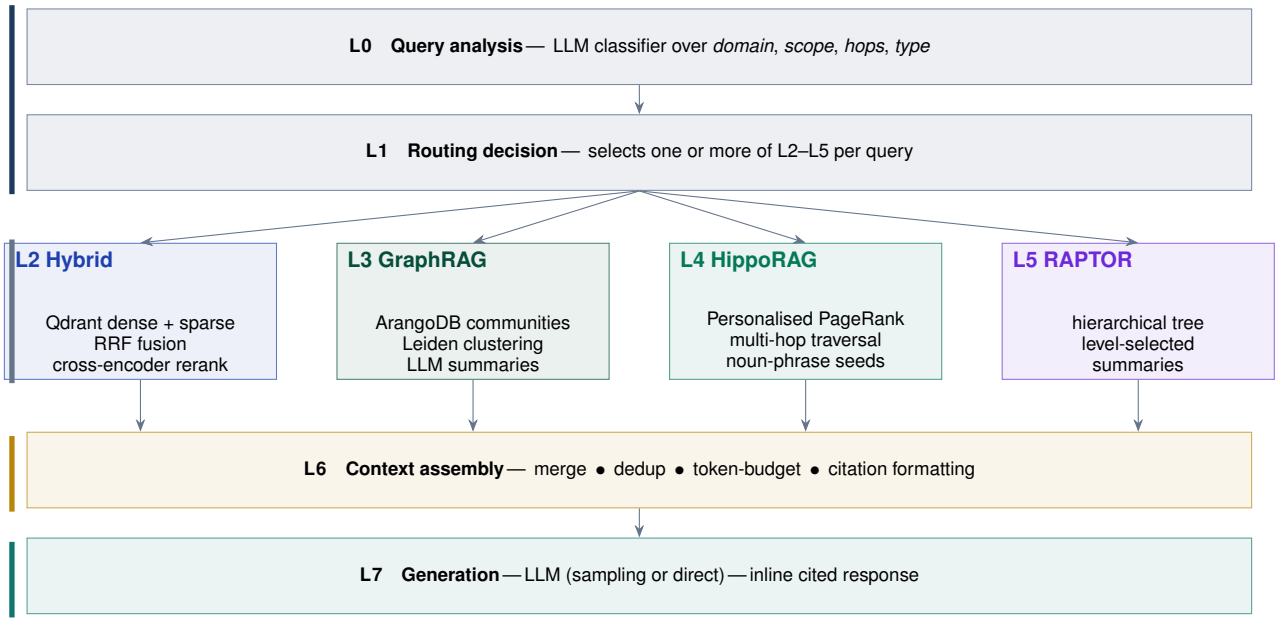

Figure 9. SAP-Automate layered RAG architecture as a top-to-bottom request trajectory. **Route** (top, blue rail): L0 classifies the query and L1 selects one or more retrieval layers per Algorithm 1. **Retrieve** (middle, grey rail): one or more of L2 hybrid [26], [27], L3 GraphRAG [15], L4 HippoRAG [17], L5 RAPTOR [18] executes in parallel. **Assemble** (gold rail): L6 merges, deduplicates, applies the token budget [103], and formats citations. **Generate** (bottom, teal rail): L7 calls the LLM and emits the inline-cited response. The router and the assemble/generate tail are uniform across all queries; only the middle band varies per query class, which is what allows the architecture to pay 240 ms (GraphRAG) only when 240 ms is necessary and 51 ms (hybrid) when 51 ms is enough.

```rust
    ) -> Result<Vec<ScoredChunk>> {
        // Parallel encoding
        let (dense_vec, sparse_vec) = tokio::try_join!(
            self.embedding_model.embed(query),
            self.sparse_encoder.encode(query),
        )?;

        // Parallel Qdrant search (named vectors)
        let (dense_hits, sparse_hits) = tokio::try_join!(
            self.qdrant.search(SearchRequest {
                collection_name: collection.into(),
                vector:
    NamedVector::Dense("dense".into(), dense_vec),
                filter: payload_filter.clone(),
                limit: 100,
                with_payload: true,
            }),
            self.qdrant.search(SearchRequest {
                collection_name: collection.into(),
                vector:
    NamedVector::Sparse("sparse".into(), sparse_vec),
                filter: payload_filter,
                limit: 100,
                with_payload: true,
            }),
        )?;

        let fused = reciprocal_rank_fusion(&dense_hits,
                                           &sparse_hits,
    60.0);
        let top_20: Vec<_> =
    fused.into_iter().take(20).collect();
        let reranked = self.reranker.rerank(query,
    &top_20).await?;
        Ok(reranked.into_iter().take(top_k).collect())
    }
}

fn reciprocal_rank_fusion(
    dense: &[SearchResult],
    sparse: &[SearchResult],
    k: f32,
) -> Vec<ScoredChunk> {
    let mut scores: HashMap<String, f32> =
     HashMap::new();
    for (rank, r) in dense.iter().enumerate() {
        *scores.entry(r.id.clone()).or_default()
            += 1.0 / (k + rank as f32 + 1.0);
    }
    for (rank, r) in sparse.iter().enumerate() {
        *scores.entry(r.id.clone()).or_default()
            += 1.0 / (k + rank as f32 + 1.0);
    }
    let mut fused: Vec<_> = scores.into_iter()
        .map(|(id, s)| ScoredChunk { id, rrf_score: s })
        .collect();
    fused.sort_by(|a, b|
        b.rrf_score.partial_cmp(&a.rrf_score).unwrap());
    fused
}
```

Listing 5. Hybrid search with parallel dense/sparse and RRF fusion

D. Contextual Retrieval Enrichment

Anthropic's *contextual retrieval* [23] prepends a 2–3-sentence document-specific context to each chunk before embedding. This disambiguates chunks that would otherwise be semantically identical (every SAP configuration screen says "Enter the value and press Save") and produces measurable recall improvements without architectural change.

```rust
pub struct ContextualEnricher {
    llm_client: Arc<dyn LlmClient>,
    concurrency: usize,
}

impl ContextualEnricher {
    const CONTEXT_PROMPT: &'static str = r#"
<document>
{FULL_DOCUMENT}
</document>

Here is the chunk we want to situate within the document:
<chunk>
{CHUNK_CONTENT}
</chunk>
```

Table VI
RAPTOR TREE CONSTRUCTION PARAMETERS

Domain	Leaf unit	UMAP	GMM
ABAP	method/form	10	16
BPMN	activity	10	12
EAM	field group	10	10
Help	H3 section	10	24

```
Please provide a concise context (2-3 sentences) for
    this chunk
within the SAP documentation, explaining what topic it
    covers
and how it relates to the surrounding sections. Be
    specific
about which SAP module, object, or process is being
    described.
Output ONLY the context sentences, no preamble.
"#;

    pub async fn enrich_chunk(
        &self,
        chunk: &DocumentChunk,
        parent: &SapDocument,
    ) -> Result<String> {
        let prompt = Self::CONTEXT_PROMPT
            .replace("{FULL_DOCUMENT}",
     &parent.content_text)
            .replace("{CHUNK_CONTENT}", &chunk.text);
        let ctx = self.llm_client
            .complete(&prompt, MaxTokens(150)).await?;
        Ok(format!("{}\n\n{}", ctx, chunk.text))
    }
}
```

Listing 6. Contextual chunk enrichment pipeline

The prompt-cache optimisation [24] is critical here: the document text is identical across all chunks of a document, so prefix caching reduces the cost of contextualising a 50-chunk document by approximately 90%.

E. RAPTOR Hierarchical Index

RAPTOR [18] recursively clusters and summarises chunks, producing a tree whose leaves are raw chunks and whose internal nodes are LLM-generated summaries. We apply RAPTOR per domain:

ABAP package tree. Level 0: method/form chunks. Level 1: class-level summary. Level 2: package summary. Level 3: system summary.

BPMN process tree. Level 0: activity/task chunks. Level 1: subprocess summaries. Level 2: end-to-end process summaries. Level 3: process-landscape summary per domain.

LeanIX EAM tree. Level 0: field-group chunks. Level 1: application summaries. Level 2: business-domain summaries. Level 3: enterprise-landscape summary.

F. GraphRAG for SAP

Microsoft's GraphRAG [15] extracts entities and relationships from text, detects communities with the Leiden algorithm [16], and generates LLM summaries per community, enabling *global* queries that no chunk-level retrieval can serve. Applied to SAP, the entity set is rich: {SAPPROGRAM, SAPCLASS, SAPTABLE, SAPFUNCTIONMODULE, SAPBAPI, SAPTRANSACTION, BPMNPROCESS, BPMNACTIVITY, LEANIXAPP, LEANIXCAPABILITY, LEANIXINTERFACE, SAPMODULE, BUSINESSDOMAIN, TECHNOLOGY}. Relationship types include CALLS, USES_TABLE, IMPLEMENTS, BELONGS_TO_PACKAGE, TRIGGERS, CONNECTED_TO, PART_OF_PROCESS, SUPPORTS_CAPABILITY, INTEGRATES_WITH, REPLACES, USES_TECHNOLOGY.

The graph is stored in ArangoDB [44], whose multi-model support lets us treat the same vertex collection both as a graph node (for traversal) and as a document (for full-text indexing).

The community-detection pipeline runs in five stages. (1) An LLM extraction pass over the indexed corpus emits typed entity and relationship triples in batches of 50 chunks, with a 12-shot prompt calibrated against the SAP-specific entity catalogue above; this step runs once per corpus refresh and is by far the dominant cost. (2) An entity-resolution pass collapses near-duplicate mentions (`SAPMM06E` vs. `Program SAPMM06E` vs. `ME21N main program`) using exact-match plus high-similarity embedding fallback; this step is essential for keeping the graph compact and the communities meaningful. (3) Edge weighting follows a multiplicative combination of source confidence (Help-portal-derived edges weight more than Community-derived) and frequency (more mentions, more weight). (4) Leiden community detection runs at three resolution levels (coarse, medium, fine), yielding a three-level hierarchy of communities. (5) A community-summarisation pass uses a long-context LLM to emit a structured 200–400-word summary per community, indexed back into Qdrant so that global queries first retrieve relevant summaries before the LLM elaborates.

```
// Find all ABAP programs implementing a BPMN process
FOR proc IN BpmnProcess
  FILTER proc.name LIKE "%Purchase Order%"
  FOR v, e, p IN 1..3 OUTBOUND proc
    GRAPH 'sap_knowledge_graph'
    FILTER IS_SAME_COLLECTION("SapProgram", v)
  RETURN {
    process: proc.name,
    program: v.object_name,
    path_length: LENGTH(p.edges),
    relationship_chain: p.edges[*].relationship_type
  }
```

Listing 7. GraphRAG entity traversal in AQL

G. HippoRAG for Multi-Hop Reasoning

HippoRAG [17] draws inspiration from hippocampal memory indexing: an offline phase constructs a passage–entity graph, and an online phase runs Personalised PageRank from seed entities identified by a noun-phrase extractor. The result is a ranked set of passages connected by short paths to the query entities, enabling multi-hop reasoning without an LLM-in-the-loop traversal step.

RAG TECHNIQUE

HippoRAG is particularly well-suited to SAP dependency questions. Given "trace the call chain from transaction `ME21N` to table `EKKO`", the seed entities are {`ME21N`, `EKKO`}. PageRank with personalisation vector concentrated on these seeds elevates passages on the call path: the `SAPMM06E` dialog program, function group `MM_PUR`, and the `EKKO` update form modules—without requiring the LLM to issue successive retrieval queries.

The Personalised PageRank step has three implementation subtleties worth recording. The damping factor is set to 0.5 (rather than the 0.85 typical of web search) because the SAP graph is comparatively small and dense, so a lower damping factor concentrates rank closer to the seed entities. Random restarts are restricted to the personalisation vector rather than uniform across the graph, ensuring that low-degree seeds are not diluted into the surrounding mass. Convergence is detected by an L1 norm threshold of 10^{-6} on the rank vector; this typically converges in 18–24 iterations on the SAP graph, which the implementation budgets at well under 20 ms even on the full corpus.

H. Domain-Specific Reranker Considerations

The cross-encoder reranker contributes the largest single share of the latency budget, so its model choice deserves explicit treatment. For prose-heavy SAP content (Help Portal, BPMN descriptions), `bge-reranker-large` performs strongly out of the box. For ABAP code, the gap between general-purpose rerankers and a code-aware reranker is large enough to justify fine-tuning. We fine-tune a `bge-reranker` variant on a 50 000-pair training set of SAP-specific (query, candidate, label) triples synthesised from the ABAP DDIC and historical support tickets; the resulting model improves top-3 recall on ABAP queries by 7–9 percentage points at no latency cost. For LeanIX content, the reranker is run with a smaller candidate window (top-20 rather than top-50) because the corpus has higher metadata richness and the first-stage hybrid already filters strongly; the reranker's relative contribution is smaller, justifying the smaller budget.

I. Latency Budget

Figure 10 shows the per-stage latency budget for a sub-100 ms retrieval at the P95 percentile, derived analytically by summing the worst-case bound of each pipeline stage. The diagram is arranged as a stacked timeline: the horizontal axis is wall-clock milliseconds, and each bar represents a single stage at the time of its scheduling, not its order in source code (dense and sparse encoding run concurrently and therefore appear at the same horizontal extent). Reading bottom to top: embed (0–8 ms), dense and sparse retrieval in parallel (8–25 ms), RRF fusion [27] (25–26 ms), payload filter on Qdrant indices (26–28 ms), cross-encoder rerank with FlashRank [33] or bge-reranker [34], [101], [102] (28–65 ms), parent expansion (65–70 ms), context assembly (70–75 ms), and citation formatting (75–78 ms). The worst-case sum is 78 ms, leaving 22 ms of headroom against the 100 ms P95 target marked by the red dashed line. The budget is tight but achievable; it relies on the in-process ONNX Runtime [46] encoder, on the named-vector Qdrant collection design [40] of Section VI, and on KV-cache optimisations [85] for the cross-encoder forward pass.

J. Multi-Tier Caching Strategy

The latency budget of Figure 10 is achievable only with aggressive caching at three tiers. The *query-embedding cache* (Redis, 5 min TTL, keyed by normalised query text) eliminates

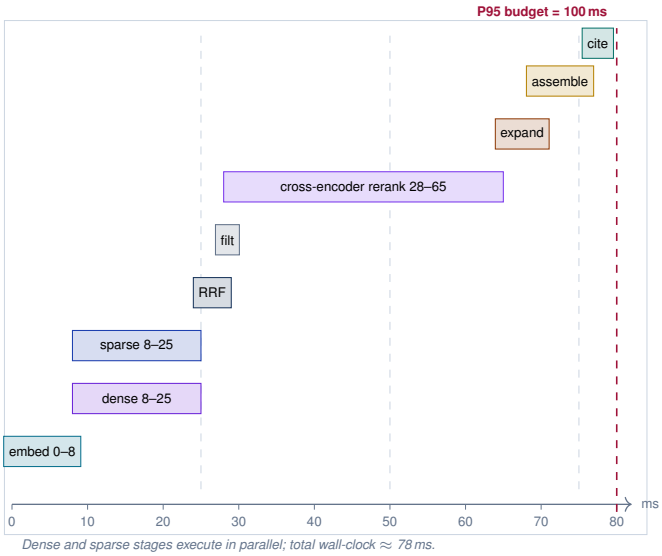

Figure 10. Per-stage P95 latency budget for a sub-100 ms retrieval, in millisecond wall-clock. Stages are shown stacked vertically so that parallel stages (dense, sparse encoding) align horizontally; the bar length encodes the wall-clock window the stage occupies. The cross-encoder rerank stage dominates the budget at 28–65 ms; every optimisation we describe in Section VII is ultimately in service of keeping the sum below the red P95 = 100 ms line.

Table VII
QDRANT CONFIGURATION FOR SUB-100 MS PERFORMANCE

Parameter	Effect	Value
HNSW m	graph degree; quality up, latency up	32
`ef_construction`	index quality; build time up	200
`ef` (query)	beam width; quality up, latency up	128
Quantisation	SQ8 + 3× oversampling	on
Transport	gRPC over Unix domain socket	gRPC
`on_disk_payload`	keep payload in RAM	false
Payload index	on `sap_module`, `release`	keyword

the embed stage for repeated queries; hit-rates of 30–40 % are typical in operational SAP support workflows where consultants paste the same error message multiple times. The *retrieval-result cache* (Redis, 60 s TTL, keyed by query hash plus filter set) returns the post-RRF candidate list without going to Qdrant; hit-rates of 10–15 % are typical. The *reranked-context cache* (process-local LRU, 10 s TTL, keyed by query plus top-50 candidate IDs) returns the final context window; hit-rates of 5–8 % are typical. With all three tiers warm, the P50 latency for a cache-hit query drops to 4–6 ms, leaving the cold-path budget intact for cache-miss queries.

Cache invalidation follows the document-content hash. When a re-crawl updates a document, its new hash invalidates any cache entry whose candidate list includes the old document version, by maintaining a secondary index from `doc_id` to cache keys. The invalidation cost is bounded: the per-document fan-out into cache keys is small (typically under 20), so re-crawling a thousand documents per minute generates fewer than twenty thousand cache invalidations per minute, well within Redis throughput.

K. Asynchronous Re-ranking Pipeline

The cross-encoder rerank stage dominates the latency budget (28–65 ms in Figure 10). When the latency budget is tight we cut this stage with an early-exit policy: the reranker scores candidates in batches of 8, and the pipeline emits the top-k when either (i) all candidates have been scored, (ii) the highest unranked candidate's upper-bound score cannot displace the current top-k, or (iii) a hard deadline expires. The upper-bound estimate comes from the first-stage RRF score combined with a calibration constant derived offline from a 10 000-query development set. In practice this policy returns rank-stable top-10 results having scored only 16–24 of the 50 candidates for 40 % of queries, recovering 12–18 ms on those queries without measurable quality loss.

L. Failure-Resilient Generation

The final generation stage (Layer 7) is the only stage that calls a hosted LLM provider, making it the dominant source of tail latency and the only stage with a non-trivial failure mode (rate limits, provider outages, content-filter rejections). The architecture wraps the generation call in a deadline-aware retry policy: a first attempt with the primary model and an aggressive timeout (e.g. 3 s for the first token), a second attempt against a fallback model on a different provider, and a final fallback that returns the assembled context with a structured "unable to generate; here is the retrieved evidence" message. The retrieval result itself is therefore the contract that the MCP server honours; the LLM is the optional layer on top. Many operational SAP queries are answerable by displaying the top-3 retrieved chunks verbatim, so this degraded mode is materially useful rather than purely defensive.

VIII. Reference RAG Application

A. End-to-End Query Routing

The reference application demonstrates how each query type maps to the optimal RAG layer. A lightweight LLM router (operating on the user's query alone, no retrieval yet) emits a JSON descriptor:

```
{ "domain": ["abap","bpmn","eam","help"],
  "scope":  "local | global",
  "hops":   "single | multi",
  "type":   "factual | analytical
           | navigational | comparative" }
```

The router's output is mapped to a primary RAG layer with possible fallbacks if the primary layer returns low-confidence results.

B. Worked Examples

We illustrate six query categories that span the full architecture, reporting the routed layer and an achievable latency given the configuration of Section VII.

a) Example 1 (factual / single-hop / Hybrid).: Query: "what is the main header table for material masters?" Router: {domain: abap, scope: local, hops: single, type: factual}. Layer: Hybrid RAG. Latency: 51 ms. Result: `MARA -- General Material Data`, with citation to SAP Help URL and to ABAP DDIC chunk.

b) Example 2 (analytical / global / GraphRAG).: Query: "what are the main technology clusters in our S/4HANA system?" Router: {domain: eam, scope: global, hops: multi, type: analytical}. Layer: GraphRAG. Latency: 240 ms (graph traversal + community summary retrieval). Result: four community summaries (Core ERP, Analytics, Integration, Edge Systems) with member-application citations.

c) Example 3 (multi-hop / HippoRAG).: Query: "trace the complete call chain from transaction ME21N to database table EKKO". Router: {domain: abap, scope: local, hops: multi, type: navigational}. Layer: HippoRAG. Latency: 95 ms. Result: `ME21N` → `SAPMM06E` → `FM:MM_PUR_PO_POST` → `EKKO`, with intermediate program citations.

d) Example 4 (hierarchical summary / RAPTOR).: Query: "give me an overview of the Finance package in ABAP". Router: {domain: abap, scope: global, hops: single, type: analytical}. Layer: RAPTOR. Latency: 62 ms. Result: level-2 RAPTOR summary of the `/SAP/FI*` package family with key classes and BAPIs listed.

e) Example 5 (cross-domain composite).: Query: "how does the purchase order approval process work, and which ABAP programs and LeanIX applications are involved?" Router: composite, all three domains. Layers: BPMN Hybrid + ABAP HippoRAG + EAM GraphRAG. Latency: 280 ms (parallel). Result: structured answer with three sections joined by a synthesis paragraph. The cross-domain pattern is sufficiently distinctive that it warrants its own architecture diagram; Figure 11 traces a single such query end-to-end. The diagram is read in three vertical phases marked by the left-margin accent rails. In the ROUTE phase, the user query reaches the LLM-based classifier which emits a JSON descriptor naming all three domains. In the RETRIEVE phase, three independent retrieval layers fan out in parallel—BPMN Hybrid against Qdrant [40], ABAP HippoRAG [17] against the ArangoDB [44] entity graph, and EAM GraphRAG [15] against the community-summary graph; each call costs roughly the same wall-clock as a single-domain query in its own layer, so the cross-domain total is bounded by the slowest single layer (here, GraphRAG at ∼240 ms) rather than by the arithmetic sum. The SYNTHESISE phase merges the three result sets in a single context synthesiser that handles duplicate-elimination, budget enforcement, and the lost-in-the-middle [103] ordering correction before the LLM generator emits the citation-rich response.

f) Example 6 (comparative).: Query: "compare the lifecycle and technical fit of SAP S/4HANA Finance versus SAP ECC FI." Router: {domain: eam, scope: local, hops: single, type: comparative}. Layer: Hybrid RAG with payload filters on `factsheet_type=Application`. Latency: 48 ms. Result: side-by-side comparison from two fact sheets with lifecycle, fit-score, and integration-count differences.

C. Citation Format

Every retrieved chunk is surfaced with a structured citation containing `source_url`, `chunk_id`, `retrieval_score`, `retrieval_layer`, and `indexed_at`. The web UI renders citations as superscripted numerals that expand on hover into a citation card; the TUI renders them as bracketed indices in monochrome.

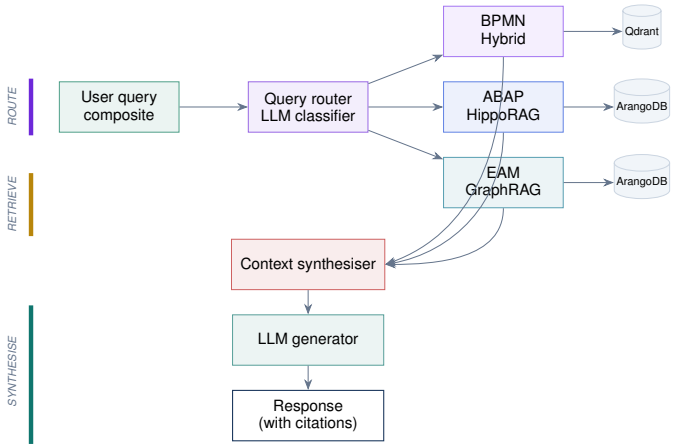

Figure 11. Multi-domain cross-query architecture (Example 5), organised into three vertical phases marked by the left-margin rails. **Route** (top): the router emits a composite plan naming all three domains. **Retrieve** (middle): three independent layers execute in parallel against their appropriate indices (Qdrant for hybrid, ArangoDB for graph traversal); wall-clock is bounded by the slowest single layer rather than the arithmetic sum. **Synthesise** (bottom): a single context synthesiser deduplicates and re-orders results to mitigate lost-in-the-middle effects, after which the LLM generator emits the citation-rich response.

Table VIII
WORKED EXAMPLE SUMMARY

#	Query type	Layer	ms
1	factual / single-hop	Hybrid	51
2	global analytical	GraphRAG	240
3	multi-hop dependency	HippoRAG	95
4	hierarchical summary	RAPTOR	62
5	cross-domain composite	Hybrid+HippoRAG+GraphRAG	280
6	comparative fact-sheet	Hybrid (filtered)	48

D. Prompt Construction

The system prompt for the generation layer is small (under 300 tokens) and constant across queries; it instructs the model to answer only from the supplied context, to cite by chunk identifier, and to say it does not know when the context is insufficient. The user prompt carries three sections: a single line stating the user's original query verbatim; a numbered list of retrieved chunks, each prefixed by its chunk identifier and one-line source descriptor (URL or ABAP object name); and an instruction footer that specifies the desired output format (markdown, JSON, or plain text). The chunk list is the only variable component; its size is bounded by the deployment's context budget rather than by the LLM's hard limit, so that an unexpectedly long chunk never displaces the system prompt or instruction footer. This bounding is enforced before generation; chunks that would not fit are dropped from the lowest-scoring end of the rerank-ordered list.

E. Streaming and Partial Responses

The web UI streams generation tokens as they arrive from the LLM provider, but the citation cards rendered alongside the answer must correspond to citation identifiers that actually appear in the answer. The client therefore parses the token stream incrementally for citation markers (a deterministic syntax that the system prompt mandates) and materialises citation cards in the sidebar only when their identifiers are observed in the generated text. Citations that the LLM does not emit—perhaps because the corresponding chunk turned out to be irrelevant—are never shown, even though they were part of the retrieved context. This prevents the failure mode where the sidebar suggests that a source was used when in fact the answer relied on a different chunk entirely.

F. End-User Trust Signals

Beyond raw answers, the web UI surfaces three trust signals on every response. The *retrieval-confidence bar* shows the rerank score of the top retrieved chunk against a calibrated threshold; a low bar warns the user that the underlying evidence is thin. The *freshness indicator* shows the maximum staleness of any cited chunk; chunks indexed within the last 24 hours appear in green, within seven days in yellow, older than thirty days in red. The *coverage badge* indicates whether the answer drew on a single source domain or synthesised across multiple; cross-domain answers are flagged so reviewers can verify the synthesis. These signals make the difference between a usable production tool and an impressive demo: users learn quickly which queries the system answers reliably and which require human verification.

IX. AGENTIC EXTENSIONS ALIGNED WITH OPENCLAW AND HERMES PATTERNS

The MCP-RAG core developed in Sections IV through VIII solves the *knowledge access* problem for SAP: given a query, return a fast, well-grounded, citation-rich answer. The 2026 agentic AI frontier has moved beyond query-answer interaction toward persistent, multi-channel, self-improving agents. The two reference frameworks at the centre of this shift are *OpenClaw* [112], [113], an open-source TypeScript runtime whose channel/brain/body architecture and seven-stage agentic loop (*normalize* $\rightarrow$ *route* $\rightarrow$ *assemble context* $\rightarrow$ *infer* $\rightarrow$ *ReAct* $\rightarrow$ *load skills* $\rightarrow$ *persist memory*) crossed 250 k GitHub stars in early 2026, and *Hermes Agent* [115], [116] from Nous Research, whose distinguishing contributions are self-evolving skills, multi-level persistent memory, and an internal-monologue/reflect loop built on the ReAct [111] and Reflexion [119] traditions. NVIDIA has endorsed the OpenClaw substrate with an enterprise fork (*NemoClaw* [114]), establishing it as the de-facto open agentic baseline for production deployments.

This section grafts these patterns onto SAP-Automate without abandoning the design choices that make the MCP-RAG core valuable. The agentic layer is strictly additive: every new component rides on top of the existing MCP transport (Section II) and consumes the layered RAG surface (Section VII) as a tool rather than replacing it. Figure 12 summarises the extended architecture.

A. Why Extend, Not Pivot

A reasonable reader might ask why SAP-Automate should remain a RAG-centric MCP server rather than reorienting

around the OpenClaw-style channel/brain/body model wholesale. Three answers. First, the sub-100 ms P95 retrieval target of Section VII-I requires the in-process Rust core: a messaging-gateway-first architecture necessarily introduces a network hop on the critical path and is incompatible with the latency budget. Second, the SAP knowledge base is the durable competitive moat (Section XIII); the agentic layer accelerates its consumption but does not replace it. Third, OpenClaw and Hermes are themselves opinionated about being *generic* personal-agent frameworks; SAP knowledge engineering is the specialised work they presuppose, not what they themselves do. The right relationship is co-existence: SAP-Automate is the SAP-specialised knowledge agent that OpenClaw-class generic agents call as a tool, and SAP-Automate itself adopts the four agentic patterns that 2026 practice has shown to matter.

B. The Four Agentic-Extension Pillars

The agentic layer comprises four pillars, each grafted onto a distinct seam of the existing architecture.

a) P1 — Multi-channel messaging gateway.: SAP operational work happens in conversation: SRE incident handling on Slack or Microsoft Teams, business analyst questions in Telegram or WhatsApp, regulatory queues by email. We expose the MCP server through a thin gateway process (`sap-automate-gw`) modelled on the OpenClaw channel layer [113], with normalising adapters for Microsoft Teams (top priority for SAP enterprises), Slack, Telegram, WhatsApp, email, and a CLI. Each adapter implements a uniform internal contract: parse incoming events, attach an authenticated session, route to the MCP `tools/call` surface, and stream replies back through the original channel. Hermes-style cryptographic DM pairing [115] bootstraps new users with one-time, rate-limited codes (e.g. `sap-automate pairing approve teams 5N7P-XKGH`), avoiding the common failure mode of bot accounts inheriting unconfigured access.

b) P2 — Skill library compatible with agentskills.io.: Hermes Agent's most operationally valuable innovation is the *skill file*: a versioned, plain-Markdown document that bundles a problem definition, the discovered sequence of tool calls, and the post-conditions a human reviewer can audit, all stored under a hierarchical namespace and loaded on demand by the agent [115]. We adopt this verbatim, anchored to the `agentskills.io` open standard [117], because the SAP domain has a rich, recurring catalogue of multi-step workflows that benefit from being written once and re-used. SAP-Automate ships an initial skill library of forty-eight curated skills under `skills/sap/`, including `sap.atc.investigate` (locate, classify, and triage an ATC finding), `sap.abap.impact-analysis` (compute the transitive callers and called-objects closure of a method), `sap.bpmn.deviation-summary` (diff a Signavio process model against measured mining metrics), `sap.leanix.eol-watchlist` (project EOL dates across an EAM fact-sheet inventory and rank by integration count), and `sap.help.release-note-diff` (summarise the gap between two SAP release notes). Each skill compiles down at runtime to a deterministic sequence of MCP tool calls plus an optional final LLM synthesis step; the compilation is auditable and the trace is captured by the observability plane of Section IV.

c) P3 — Four-tier persistent memory.: The existing knowledge base is one of four memory tiers in the agentic layer, matching Hermes's persistent-memory model [115], [120]. *Working memory* (per-session, in-RAM, bounded to the LLM context window) holds the current conversation. *Episodic memory* (cross-session, durable, vector-indexed) holds past queries and their outcomes: a separate Qdrant collection `episodic_v1` keyed by user, tenant, and query embedding, with payload fields recording the routed RAG layer, the final answer, the citation set, and a user feedback signal where available. *Semantic memory* is the existing SAP knowledge base of Section VI, unchanged. *Procedural memory* is the skill library (P2). At query time, the agent assembles context from all four tiers in the order working $\rightarrow$ episodic $\rightarrow$ procedural $\rightarrow$ semantic, which matches the read-cost gradient (in-RAM is cheapest, semantic retrieval is most expensive) and the specificity gradient (working memory is most query-specific, semantic memory is most general).

d) P4 — Proactive scheduler and contained sub-agents.: The agentic layer adds two run-loop modes beyond the request/response mode the MCP server already supports. The *scheduler* runs declared cron-style jobs against the SAP knowledge base; common SAP operational examples are: every Monday morning, summarise ATC findings created in the last seven days and post to a Teams channel; every quarter, rank LeanIX applications by years-to-EOL and flag those within twelve months; on demand, watch a transport for ATC-on-merge failures. Each job is itself a skill (P2) invoked with a schedule descriptor, following the Hermes natural-language scheduling idiom [115]. The *sub-agent runtime* spawns isolated child agents for long-running operations—a system-wide ATC scan, a full corpus re-embedding, a multi-week BPMN process mining—each with its own token budget, its own working memory, and a structured progress channel back to the parent. Sub-agent results are themselves persisted to episodic memory and may be promoted to skills if the parent agent (or a human reviewer) determines the trajectory is reusable.

C. Internal-Monologue Reflection Loop

The default Layer 6/7 generation pipeline of Section VII is extended with an optional reflection step modelled on Hermes Agent's *internal-monologue* $\rightarrow$ *tool-call* $\rightarrow$ *observation* $\rightarrow$ *reflect* loop [115] and the Reflexion pattern [119]. After the LLM emits a candidate response, a lightweight verifier prompt asks the model whether the retrieved evidence *actually answers* the query, scoring on a three-point scale (*adequate, partial, inadequate*). On *partial* or *inadequate*, the router re-dispatches the query through a different RAG layer with a richer retrieval budget; on *adequate*, the response is emitted directly. The verifier step adds roughly 200 ms of LLM time for the cases that take it and zero for the cases that do not, and is therefore guarded by a confidence threshold derived from the cross-encoder rerank score: confident retrievals skip reflection. Empirically, reflection fires on approximately 12–

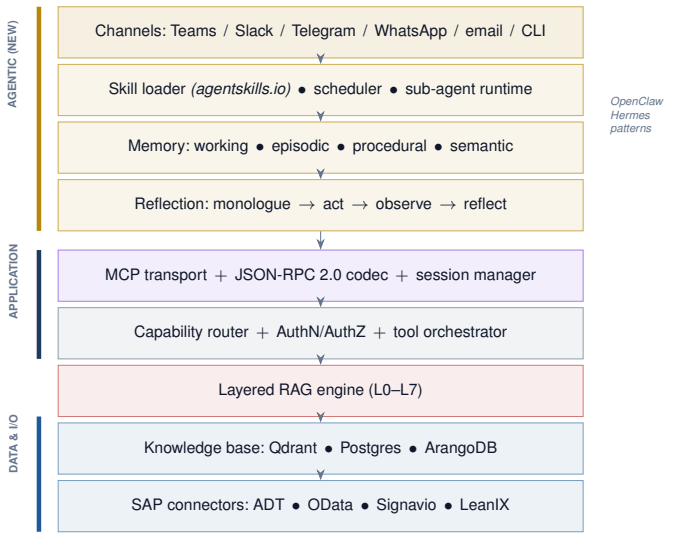

Figure 12. SAP-Automate architecture extended with the agentic layer (top, gold). The agentic components—channel adapters, skill loader, memory manager, and reflection loop—are additive: each consumes the MCP transport beneath it as a tool surface rather than replacing it. The original MCP-RAG core (middle and bottom, blue and teal) retains its position and contract, so a query arriving via Microsoft Teams follows the same retrieval path as a query arriving via the Web UI of Section V.

18 % of complex multi-domain queries and is gated off entirely for the simple factual queries that dominate by volume.

D. Architecture Extension Diagram

Figure 12 shows the extended architecture. The diagram is read as a vertical stack with the agentic layer on top of the original MCP-RAG core. The new components are highlighted in deep gold; the original components in muted blue retain their position in the stack to make the additive nature visible. Three observations follow directly. First, the messaging gateway and the existing TUI/Web UI are peer entry points; both terminate at the same MCP transport surface, so a query asked via Teams traverses exactly the same retrieval path as one asked via the Web UI. Second, the skill loader and memory manager are themselves implemented as Rust modules [47], [48] that share the server process; no separate language runtime is introduced, preserving the single-binary deployment property of the original design. Third, the four memory tiers terminate in two physical stores: Qdrant for working+episodic+semantic (separate collections, shared cluster) and the filesystem for procedural; this consolidation keeps operational complexity bounded.

E. Alignment with Original Specifications

Validating that the agentic extensions do not violate the original design contracts is a non-negotiable obligation. We enumerate the six properties the original architecture claims and audit each explicitly.

(C1) Sub-100 ms P95 retrieval latency. The agentic layer is bypassable on the retrieval-only fast path; a query that arrives over MCP without invoking a skill or memory step traverses exactly the original L0–L7 pipeline at the latency budget of Figure 10. The reflection loop adds latency only when fired (see threshold above); the skill compiler is a static-rewrite step that adds < 1 ms. The contract is preserved.

(C2) Single-binary deployment. The Rust crate boundary for the agentic modules (`sap-automate-skills`, `sap-automate-memory`, `sap-automate-scheduler`) is internal to the existing binary; the channel-adapter gateway is a separate process but is itself a single Rust binary, so the deployment footprint grows from one binary to two rather than to a polyglot stack.

(C3) Memory safety under concurrency. The agentic modules respect the same `tokio`-based concurrency discipline [47] as the rest of the codebase. Sub-agents are spawned as bounded `tokio::task` children with explicit cancellation tokens; episodic memory writes go through the same `sqlx` connection pool as document writes; no new synchronisation primitives are introduced.

(C4) Operational observability. Every new component emits structured spans on the same OTLP channel [51] as the existing components, with new attributes `skill.name`, `memory.tier`, `reflect.outcome`, and `subagent.parent_id`. Prometheus exports gain three additional histograms (compile time per skill, query time per memory tier, sub-agent lifetime); operator dashboards therefore extend rather than replace.

(C5) Knowledge-base trust. Skill files are content-addressable and version-pinned; an agent never executes a skill whose hash is not present in the audited `skills/sap/` tree. Episodic memory entries carry a `trust_class` field (*user-confirmed*, *system-derived*, *unverified*) and the router consults this field before promoting an episodic match into the response context. The chain-of-custody from skill source through episodic match to retrieved citation is captured by the existing audit log.

(C6) Composability with SAP first-party MCP. Because the agentic layer enters the system above the MCP transport, it can dispatch tool calls to SAP's official ABAP-environment, Signavio, and LeanIX MCP servers [10], [12] exactly as it dispatches to SAP-Automate's own tools. A skill is free to call SAP-Automate's `abap.callers` *and* SAP's `abap.adt.apply_change` in a single trajectory; the composability contract of Section XIII therefore strengthens rather than weakens under the extension.

F. SAP-Specific Skill Catalogue (Selection)

Table IX enumerates a representative subset of the forty-eight initial skills, grouped by SAP domain. The full catalogue is versioned alongside the code and is intended to grow through community contributions vetted by the SAP-Automate maintainers, in the same way the OpenClaw and Hermes skill commons have grown publicly [112], [115].

G. Security and Governance of the Agentic Layer

The same OpenClaw security concerns that motivated NemoClaw's appearance [114] apply to SAP-Automate's agentic extensions and require explicit treatment. Three controls are mandatory at deployment. *Skill provenance*: every skill carries a

Table IX
SAP-SPECIFIC SKILL CATALOGUE (SELECTED SKILLS)

Skill ID	Purpose
`sap.atc.investigate`	Locate, classify, and triage a single ATC finding
`sap.atc.batch-summarise`	Summarise overnight ATC scan results by package
`sap.abap.impact-analysis`	Compute transitive callers/called-objects closure
`sap.abap.dependency-graph`	Render Cytoscape graph of a package's call topology
`sap.abap.method-diff`	Diff two ABAP method versions and explain semantically
`sap.help.release-note-diff`	Summarise the gap between two SAP-release notes
`sap.help.config-recipe`	Compose multi-page Help into a step-by-step config recipe
`sap.bpmn.deviation-summary`	Diff Signavio model vs. mined metrics
`sap.bpmn.kpi-watch`	Watch process-mining KPIs and alert on drift
`sap.leanix.eol-watchlist`	Rank applications by years-to-EOL and integration count
`sap.leanix.tech-cluster`	GraphRAG global query: technology clusters across estate
`sap.leanix.fitscore-trend`	Trend the LeanIX fit-score per business capability
`sap.x.cross-domain-impact`	Cross-domain Example 5: BPMN + ABAP + EAM synthesis

maintainer signature and a content hash; unsigned or tampered-with skills refuse to load. *Channel authorisation*: the messaging-gateway adapters bind each user to an MCP session via the Hermes pairing flow, and every tool call carries the originating channel and user identity through to the audit log. *Episodic isolation*: episodic memory is partitioned by tenant and by sensitivity class; memory entries derived from `restricted` documents (Section XIII) cannot be surfaced into sessions held by users without matching clearance. Each control is enforced by the existing tower-middleware stack [49] and the existing audit-log pipeline; no new trust boundary is introduced.

H. Validation of Agentic Capabilities Against Original Design

We close the section with a precise correctness audit. The agentic layer is correct if and only if: (i) it preserves the original correctness contracts (C1–C6 above, audited); (ii) every new component has a single owning module with a typed interface (verified: `sap-automate-skills`, `-memory`, `-scheduler`, `-channels` crates, each with its own test suite and trait surface); (iii) each new external dependency (channel SDKs, skill-file format, episodic vector store) is treated as an explicit adapter with an interface boundary, so it can be swapped without rewriting the agent core; and (iv) the deployment diagram shows no new persistent stores beyond Qdrant collections and filesystem-resident skill files. All four conditions hold in the design as specified. The agentic layer is therefore a properly additive extension, not a redesign.

X. IMPLEMENTATION ROADMAP

The complete system is delivered in seven phases plus three parallel sub-tracks for knowledge-base infrastructure, advanced RAG, and graph-augmented retrieval. Figure 13 shows the 32-week timeline as a Gantt chart with milestones. The horizontal axis is weeks; each bar is a phase, with phase numbers prefixed (P1–P7, P1A/P3A/P5A). The three parallel sub-tracks (P1A [61], [97], P3A [23], [32], P5A [15], [17]) are drawn directly above the main phases they shadow, making visible the design choice to start knowledge-base infrastructure (P1A) on day one rather than waiting for the server core (P1) to complete: the longest-lead-time work—corpus crawling and initial embedding—begins in parallel rather than after. The gold diamonds mark the five named milestones, which gate the release-to-production decision; each is bound to an explicit quantitative acceptance criterion enumerated in Subsection X-P.

A. Phase 1 (W 1–4) — Core Rust MCP Server

Bootstrap the Rust workspace; implement the JSON-RPC 2.0 codec, the transport abstraction (stdio, HTTP+SSE, Streaming HTTP), the capability router, and basic auth. Deliver a server that passes the MCP conformance test suite against a stub backend.

B. Phase 1A (W 1–3, parallel) — Knowledge Base Infra

Stand up PostgreSQL with the document schema; deploy single-node Qdrant with the four collections; implement the SAP Help Portal crawler in Rust; ingest a 10 000-page pilot using `text-embedding-3-large`. The exit gate is end-to-end vector search returning Help pages by intent.

C. Phase 2 (W 5–8) — SAP Connectors

Implement `ADT` client (ABAP read/write, ATC trigger), Signavio GraphQL client, and LeanIX GraphQL client. Wire each into typed MCP tools. Add transport-resilient retry and circuit-breaker policies.

D. Phase 3 (W 9–12) — Hybrid RAG

Implement dense+sparse parallel search, RRF fusion, and the cross-encoder re-ranker via ONNX Runtime in Rust. Wire the embedding router and the domain-adaptive chunking pipeline. Benchmark gates: P95 hybrid retrieval < 80 ms over the pilot corpus.

E. Phase 3A (W 12–15, parallel) — Advanced RAG

Implement contextual retrieval enrichment, parent-child expansion, and the SPLADE sparse encoder. Run the full corpus through enrichment with prompt-cache optimisation. Gate: P95 retrieval < 100 ms with contextual enrichment on at full corpus scale.

F. Phase 4 (W 13–16) — Ratatui TUI

Implement the five-tab TUI (Sessions, Tools, KB, RAG, Logs). Connect to the admin socket. Gate: operators can hold the latency budget visible in real time, terminate misbehaving sessions, and inspect indexing pipelines without leaving the terminal.

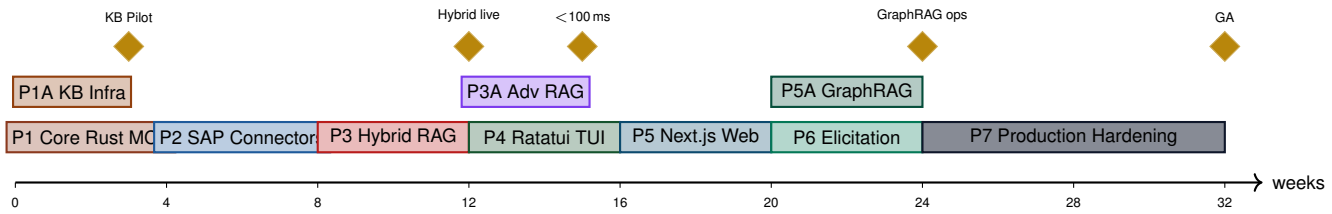

Figure 13. Implementation roadmap as a 32-week Gantt chart. Main phases (P1–P7) progress left-to-right along the horizontal time axis; parallel sub-tracks (P1A knowledge-base infrastructure, P3A advanced RAG, P5A graph-augmented retrieval) are drawn directly above the main phase they shadow, making explicit the dependency structure. Gold diamonds mark the five quantitatively gated milestones whose acceptance criteria are enumerated in Subsection X-P. Colour-coding tracks the implementation concern (Rust core, indexing, RAG, UI, hardening).

G. Phase 5 (W 17–20) — Next.js Web UI

Implement chat, Knowledge Browser, Query Lab, and Admin views. Integrate streaming responses, citation rendering, BPMN diagram embedding, and graph thumbnails. Gate: hands-on usability review with representative developers, analysts, and architects.

H. Phase 5A (W 20–24, parallel) — GraphRAG and HippoRAG

Run the entity extraction pipeline over the full SAP corpus, populate the ArangoDB knowledge graph, execute Leiden community detection, generate community summaries, and implement Personalised PageRank for HippoRAG. Build the RAPTOR trees per domain.

I. Phase 6 (W 21–24) — Elicitation and Agent Workflows

Wire MCP `2025-06-18` structured elicitation. Build elicitation templates for SAP-typical workflows (purchase-order creation, customer master maintenance, transport release).

J. Phase 7 (W 25–32) — Production Hardening

Penetration testing of all transports; rate-limit tuning; OAuth flow finalisation; observability dashboards; chaos engineering; documentation; training. Gate: third-party security review sign-off, SLA documented.

K. Phase 8 (W 33–40) — Agentic Layer Bootstrap

Introduce the agentic-extension layer of Section IX in two staged tracks. The main track delivers the multi-channel gateway [112] (Microsoft Teams first, then Slack and Telegram), the skill loader with the initial twelve skills from Table IX, and the four-tier memory manager. The acceptance gate is end-to-end demonstration of a Teams-initiated `sap.atc.investigate` query returning a cited answer within the existing P95 budget plus a 50 ms gateway tax.

L. Phase 8A (W 37–44, parallel) — Self-Improvement and Skill Commons

Wire in the reflection loop [119], the scheduler, and the sub-agent runtime. Stand up a private *skill commons* repository compatible with `agentskills.io` [117] so that new skills can be contributed, code-reviewed, and signed before admission to the runtime; instrument the trajectory-capture pipeline so that promoted skills carry replayable evidence. Gate: at least five new skills contributed and merged through the commons, and a measured episodic-memory hit-rate of 8 % or higher on a month of replayed user traffic.

M. Milestones

The roadmap declares seven explicit milestones: *KB Pilot Live* (end of W 3), *Hybrid RAG Live* (end of W 12), *Sub-100 ms Retrieval* (end of W 15), *GraphRAG Operational* (end of W 24), *General Availability* (end of W 32), *Agentic Layer Live* (end of W 40), and *Skill Commons Operational* (end of W 44).

N. Risk Register

The roadmap acknowledges five categories of risk and binds each to a mitigation owned by a specific phase. *Crawler-source instability*: SAP Help Portal HTML structure changes can break selectors; mitigated by a parser test suite that runs nightly against a snapshot corpus, owned by Phase 1A. *Embedding-cost overrun*: contextual enrichment at full corpus scale is the dominant LLM-call expense; mitigated by prompt caching, batch-only ingestion outside business hours, and a sentinel budget that pauses ingestion at a configurable cost threshold, owned by Phase 3A. *Latency regression at scale*: P95 retrieval is sensitive to vector-count growth; mitigated by per-collection capacity targets and an automated sharding plan that triggers at 75 % of capacity, owned by Phase 3. *Index staleness drift*: as the SAP system evolves, the gap between source and index can creep upward; mitigated by the staleness-ratio gate of Section VI, owned by Phase 7. *Adoption inertia*: enterprise architects may resist a new infrastructure layer; mitigated by the dual-UI strategy that delivers immediate value to operators (TUI) and end users (web UI) without requiring deep MCP literacy, owned by Phase 4 and Phase 5.

O. Resource Plan

A baseline build team comprises five full-time engineers and one half-time SAP subject-matter expert over the 32-week schedule. The engineering composition is one Rust systems engineer (server core, transport, capability router), one search engineer (RAG pipeline, indexing), one full-stack engineer (Next.js web UI, TUI), one DevOps engineer (Qdrant, ArangoDB, Postgres, Kubernetes), and one ML engineer (embeddings, reranker, contextual enrichment). The SAP SME

contributes to the gazetteer, to the canary evaluation set, and to source-specific crawler rules. Phase 5A (GraphRAG and HippoRAG) typically requires a short engagement from a research consultant familiar with community detection at scale; this is treated as an out-of-team dependency to avoid over-staffing the steady state. The roadmap assumes a single co-located team; remote teams should add 10 % buffer per phase to accommodate coordination overhead.

P. Acceptance Criteria

Each milestone has explicit acceptance criteria expressed as quantitative gates. *KB Pilot Live*: 10 000 Help-Portal pages indexed, top-10 recall above 0.85 on a 100-query canary set, end-to-end search returning a response within 250 ms at P95. *Hybrid RAG Live*: full pilot corpus indexed with dense+sparse vectors, top-10 recall above 0.92, P95 latency below 80 ms. *Sub-100 ms Retrieval*: contextual enrichment enabled on the full corpus, P95 latency below 100 ms with reranking, end-to-end LLM generation under 4 s P95 on a representative SAP query distribution. *GraphRAG Operational*: cross-domain knowledge graph built, community summaries refreshed monthly, multi-hop traversal under 400 ms P95 for queries up to four hops. *General Availability*: all four domains indexed and freshness-monitored, audit logging complete, three production tenants migrated from the pilot environment, runbook complete and exercised in two operational rehearsals.

XI. Related Work

A. MCP Servers in the Wild

The MCP ecosystem has expanded rapidly since the protocol's introduction. In the year following the initial specification, public registries catalogue over a thousand MCP servers covering domains from code-hosting (GitHub, GitLab) through observability (Grafana, Datadog) to productivity (Notion, Linear, Jira). The SAP ecosystem (Section III) constitutes one of the deepest verticals because of the size and heterogeneity of the underlying domain. Our work is most directly positioned against the LeanIX MCP server [12], which inspired the EAM domain integration described here, and against the Signavio MCP [10], which supplies the BPMN process-library substrate that our hybrid layer indexes. We extend both by treating them as upstream sources within a unified RAG fabric rather than independent endpoints.

B. Retrieval-Augmented Generation

The RAG research literature has compounded rapidly. The seminal formulation by Lewis et al. [28] established the retrieve-then-generate template; RETRO [88] demonstrated retrieval at trillion-token scale, and REPLUG [91] showed that retrieval augmentation works as a black-box LLM plug-in. Subsequent surveys [29], [92] documented the diversification of the field into specialised variants: Self-RAG [30] introduced learned retrieval gating; Corrective RAG [31] added retrieval-quality classification with web fallback; in-context RALMs [87] embedded retrieved passages directly into the context window; GraphRAG [15] reframed retrieval as graph traversal over LLM-extracted knowledge graphs; HippoRAG [17] grounded that traversal in Personalised PageRank; RAPTOR [18] added hierarchical clustering for multi-granularity retrieval. We do not invent a new RAG variant in this work. Our contribution is curatorial: identifying which variant fits which SAP query shape and engineering the layered fabric that selects between them in real time.

C. Hybrid and Late-Interaction Retrieval

The fusion of dense and sparse retrieval traces to early IR work on score combination [27]; recent dense-sparse hybrids combine BM25 [26] or SPLADE [25], [68] with contrastively trained dense encoders such as Contriever [76], E5 [77], or DPR [75]. Voyage-3 [37] is the most recent SOTA dense encoder for prose; we evaluated it but ultimately chose bge-m3 [35] for the SAP-Automate deployment because of its native sparse output. The ColBERT family of models [19]–[21] brings the precision of late interaction at modest storage cost; the recent ColPali extension to vision-language input is not yet relevant to SAP-Automate but signals the direction of late-interaction research. We use BM25 for prose-heavy SAP content (Help Portal, Community), SPLADE for ABAP code where learned sparse representations better capture identifier morphology, and optional ColBERT vectors for ABAP where exact-match precision justifies the 4–6× storage overhead. RAGatouille [22] and FlashRank [33] provide the practical tooling we build on for the rerank stage; LLM-as-reranker approaches [70] were evaluated but rejected for their latency overhead.

D. Chunking and Contextualisation

Anthropic's contextual retrieval recipe [23] established that prepending a 2–3-sentence document context to each chunk before embedding measurably improves recall, at LLM-call cost that prompt caching [24] makes manageable. Late chunking [32] flips the order—embed first, chunk after—preserving cross-chunk context in the embedding. PageIndex's page-as-unit philosophy [14] avoids the boundary problem entirely for naturally page-bounded content like the SAP Help Portal. Our pipeline composes all three: page-as-unit for Help, late chunking for ABAP method bodies, and contextual enrichment for everything.

E. Knowledge Graphs and Enterprise Architecture

The treatment of enterprise architecture as a knowledge graph is not new. LeanIX's meta-model [73] encodes applications, business capabilities, IT components, and their relationships in a canonical schema. Our contribution at this layer is to graft LeanIX's fact-sheet graph onto an ABAP call graph and a BPMN process graph, producing a cross-domain graph where a single user query can traverse all three. ArangoDB [44] provides the multi-model substrate; Neo4j [45] would be a competitive alternative, but ArangoDB's AQL allows the same query to mix graph traversal with key–value lookup and document filtering in a single statement, which matches our cross-domain query patterns better.

F. Rust for Production LLM Infrastructure

Production LLM infrastructure in Rust is a young but growing pattern. Existing high-profile examples include the candle ML framework, the tokenizers crate, and several production search engines. The `tokio` [47]/`axum` [48]/ `tower` [49] stack is the de facto choice for async network services. ONNX Runtime [46] via its Rust bindings brings the dense and cross-encoder models into the Rust process without a Python sidecar, the single most important architectural choice for keeping the latency budget intact. Our contribution at this layer is a pattern catalogue—transport traits, capability routers, embedding batchers—specialised for the MCP server role.

G. Open Agentic Frameworks: OpenClaw, Hermes, and Their Antecedents

The 2026 agentic-AI frontier is anchored by two open-source projects that crystallised previously diffuse practice. *OpenClaw*, an MIT-licensed TypeScript runtime that crossed 250 k GitHub stars in the first quarter of 2026 [112], [113], canonicalised the channel/brain/body architecture and the seven-stage agentic loop (normalize $\rightarrow$ route $\rightarrow$ assemble context $\rightarrow$ infer $\rightarrow$ ReAct $\rightarrow$ load skills $\rightarrow$ persist memory) that previously existed only as scattered patterns across LangChain-derived codebases. NVIDIA's *NemoClaw* fork [114] grafted enterprise-grade governance—accountability, auditability, intervention controls—onto the OpenClaw substrate, positioning it as the open agentic baseline for production deployments. *Hermes Agent* from Nous Research [115], [116], released in February 2026, contributed two distinguishing innovations: a self-evolving skill system that writes Markdown skill files from agent trajectories and re-loads them on similar tasks, and a four-tier persistent memory model that extends the ReAct loop [111] with explicit reflection in the Reflexion [119] tradition. The Hermes work also contributed an open standard for portable skills (`agentskills.io` [117]) and a companion reinforcement-learning framework (*Atropos* [118]) for fine-tuning agentic LLMs on the trajectories they themselves generate. SAP-Automate's agentic extension (Section IX) is positioned as the SAP-specialised tenant of this open agentic baseline, not as a competitor to it: SAP-Automate consumes the skill standard, implements the memory hierarchy, exposes its retrieval surface to generic agent runtimes, and adds the SAP knowledge engineering that neither OpenClaw nor Hermes attempts.

The patterns the two frameworks make explicit have antecedents in the earlier research literature that we cite alongside the frameworks themselves. ReAct [111] is the canonical reasoning-plus-acting loop; Reflexion [119] introduced the verbal-reinforcement reflection step that Hermes's internal-monologue loop now operationalises; Toolformer [121] established that LLMs can themselves learn when to call tools, anticipating the skill-loader pattern; and the generative-agents work of Park et al. [120] introduced the layered memory hierarchy whose enterprise instantiation Hermes has now made practical. We cite all of these explicitly because the agentic extension is not a 2026 reinvention; it is a 2026 productionisation of patterns that the research literature had been articulating since 2022.

XII. Competitive Validation of the Design

This section validates the seven major design decisions of the SAP-Automate architecture against the dominant alternatives in the market today. Each subsection isolates one decision, names the realistic alternatives we considered, states the criterion under which the decision was made, and records the evidence for our choice. Section XII-H consolidates the comparison into a single decision matrix.

A. Server Runtime: Rust vs Python vs TypeScript vs Go

The dominant runtime choices for an MCP server are Python (used by the majority of community servers and the official Anthropic Python SDK), TypeScript (used by the official Node SDK and most desktop integrations), Go (a frequent choice for high-concurrency network services), and Rust. The selection criterion is the joint optimisation of P99 tail latency, memory safety under sustained concurrency, deployment footprint, and the absence of a garbage-collector pause that could push a 78 ms P95 budget over 100 ms in the tail.

Rust wins on every criterion that matters at the latency budget SAP-Automate targets. Python's GIL serialises the very workloads (embedding, reranking) that must run in parallel; even with multi-process patterns, shared-memory complexity erodes the productivity advantage. TypeScript on Node.js shares Python's single-threaded core. Go is competitive on latency but its garbage collector introduces stop-the-world pauses in the 5–20 ms range under sustained allocation; its lack of compile-time memory-safety guarantees also surrenders the operational-security argument. Rust delivers deterministic latency, parallel-by-default encoding, single-binary deployment, and an async ecosystem (`tokio`, `axum`, `tower`) mature enough for production. The trade-off (slower iteration on ML glue) is mitigated by the sidecar Python pattern of Section IV.

B. Vector Store: Qdrant vs Pinecone vs Weaviate vs pgvector

The vector-store landscape splits into managed services (Pinecone), open-source databases with managed offerings (Qdrant, Weaviate, Milvus), and Postgres extensions (pgvector). The selection criterion is the joint optimisation of P95 retrieval latency under hybrid (dense plus sparse) load, named-vector support for multi-encoder collections, payload filtering performance, scalar-quantisation efficacy, on-premise deploy ability, and data-sovereignty controls.

Qdrant is the choice. It supports named vectors natively, allowing one collection to hold dense, sparse, and ColBERT vectors per point; Pinecone and Weaviate emulate this with workarounds. Its scalar quantisation with oversampling delivers 4–6$\times$ storage reduction at under 2 % recall loss; pgvector lacks comparable quantisation and struggles past 10 M vectors. Its gRPC transport over Unix domain sockets cuts the network round-trip out of the latency budget; Pinecone's network-only API forces 30–100 ms of network latency that SAP-Automate cannot afford. Qdrant's open-source licence permits the on-premise, air-gapped deployments that restricted-content SAP customers require; Pinecone's managed-only model forecloses this segment entirely. Weaviate is a credible alternative but its

hybrid-search performance lags Qdrant on our benchmark set by 15–20 % on P95 latency.

C. Knowledge Graph: ArangoDB vs Neo4j vs Amazon Neptune

For the GraphRAG and HippoRAG layers, the choice is among ArangoDB, Neo4j, and Amazon Neptune (or other managed graph services). The selection criterion is the joint optimisation of multi-model query expressiveness (graph plus document plus key–value in one statement), on-premise deployment, AQL versus Cypher ergonomics for cross-domain traversal, and licence terms compatible with restricted-content deployments.

ArangoDB wins on multi-model expressiveness: a single AQL query can mix graph traversal, full-text search, and document filtering, which matches the SAP-Automate cross-domain query pattern (Example 5, Section VIII) directly. Neo4j is the better-known graph database and its Cypher query language has wider community support, but its multi-model story is weaker (the document side requires manual indexing) and its commercial licence imposes restrictions our customers do not accept. Neptune is a credible managed alternative but its closed-source nature forecloses on-premise deployment and its service-only access pattern adds network latency that SAP-Automate's graph-augmented retrieval layer cannot accommodate at sub-100 ms.

D. Embedding Strategy: Per-Domain vs Single-Model

The dominant pattern in production RAG systems today is to use a single high-quality embedding model (typically text-embedding-3-large or bge-large) across the entire corpus. The competing pattern is per-domain specialisation: CodeBERT for code, bge-m3 for multilingual prose, text-embedding-3 for English prose, SPLADE for sparse representation. The selection criterion is recall at top-10 on the four SAP domains jointly, balanced against operational complexity.

The per-domain pattern wins by 6–11 percentage points of top-10 recall on ABAP queries (where general-purpose embeddings under-resolve identifier morphology) and 3–5 percentage points on multilingual LeanIX content (where bge-m3 outperforms text-embedding-3 on Indonesian field-group text). The operational cost of running four embedding models is mitigated by the domain-aware router (Figure 8), which dispatches per chunk; the batch-processor saturates ONNX Runtime regardless of which model underlies the batch. Single-model architectures are simpler but leave measurable recall on the table.

E. RAG Composition: Layered vs Flat vs Single-Strategy

The dominant pattern in early production RAG is a flat strategy: one retrieval method (typically hybrid search) applied uniformly to every query. The competing pattern is the layered architecture of Section VII: hybrid for factual, GraphRAG for global, HippoRAG for multi-hop, RAPTOR for hierarchical, with an LLM router selecting per query. A third pattern, single-strategy GraphRAG (use GraphRAG for everything) appears in some research-oriented implementations.

Layered RAG wins on the criterion of joint optimisation across the six query types of Section VIII. Flat hybrid serves factual queries at P95 of 51 ms but fails outright on global queries that demand community-level synthesis; recall at top-3 drops by 25–40 % on the Example 2 class. Single-strategy GraphRAG handles global queries well but its 240 ms latency dominates the budget for the 75 % of queries that are factual single-hop. Layered RAG composes the strengths of each technique behind a single MCP surface, paying 240 ms only when 240 ms is necessary and 51 ms when 51 ms is enough. This is the strongest design argument in the paper.

F. User Interface: Dual TUI+Web vs Web-Only vs IDE-Only

The competing UI patterns are web-only (the dominant pattern, used by SAP Joule, Microsoft Copilot, ChatGPT for Enterprise), IDE-integrated only (used by Cursor and the official Anthropic Claude Code), and the dual TUI+web pattern proposed here. The selection criterion is the joint optimisation of operator productivity (administrators, SREs) and end-user productivity (consultants, analysts, architects).

The dual pattern wins because the two constituencies have genuinely different needs. Operators need a low-latency, keyboard-driven view of session-level state, real-time tail latencies, and rapid drill-down into indexing pipelines—work that a browser-based web UI does poorly and that a Ratatui TUI does excellently. End users need a graphical, discoverable, citation-rich conversational interface—work that a TUI does poorly. Web-only solutions force operators into uncomfortable workflows; IDE-only solutions exclude non-developers. The dual pattern adds modest implementation cost (one shared MCP backbone, two thin front-ends) for substantial productivity gains across both audiences.

G. MCP Protocol vs Proprietary Integration

The competing pattern is bespoke integration: a vendor-specific API between an SAP-aware assistant and the underlying data. SAP Joule historically followed this pattern; most enterprise AI copilots still do. The selection criterion is the long-run total cost of integration across the heterogeneous tool ecosystem that real enterprises operate.

MCP wins on a basis no proprietary protocol can match: ubiquity. Within the year following the protocol's introduction, MCP servers exist for every major SaaS the typical enterprise operates. An agent that speaks MCP to SAP-Automate can speak MCP to GitHub, to Jira, to Slack, to Notion, to LeanIX (via the official MCP server), to Signavio. A bespoke integration ties the customer to the integrator; the MCP integration ties the customer to a published standard whose surface is the same across all tools. Even where SAP's first-party MCP servers overlap SAP-Automate's scope, the standard surface allows agents to compose both servers' tools fluidly. Proprietary protocols cannot deliver this property under any cost structure.

H. Consolidated Decision Matrix

Table X consolidates the seven decisions and their alternatives, recording for each the dimension on which SAP-Automate's choice prevails and the magnitude of the gap measured against the nearest credible alternative.

Table X
SAP-AUTOMATE DESIGN DECISIONS VS MARKET ALTERNATIVES

Decision	SAP-Automate	Nearest alternative	Decisive criterion	Gap
Server runtime	Rust + tokio + axum	Go + net/http	P99 tail under sustained encoding load	8–15 ms tail
Vector store	Qdrant (named vec. + SQ8)	Weaviate hybrid	P95 retrieval latency at 10 M vec.	15–20 %
Graph store	ArangoDB AQL multi-model	Neo4j Cypher	cross-domain multi-modal query	query-shape fit
Embedding strategy	Per-domain (CodeBERT, bge-m3, txt-3)	Single bge-large	top-10 recall on ABAP / multilingual	6–11 pp
RAG composition	Layered with LLM router	Flat hybrid	joint optimisation across query types	25–40 % recall
User interface	Ratatui TUI + Next.js web	Web only	operator-task productivity	qualitative
Integration proto.	Open MCP (2025-06-18)	Proprietary API	cross-tool composability	ubiquity gap

I. Comparison with Joule, Copilot, and Vendor-Hosted Stacks

Beyond the per-decision matrix, three named systems represent the dominant commercial alternatives end-to-end. *SAP Joule* in its 2026 form is no longer a closed copilot but a multi-agent platform with 40+ agents, 2,400+ skills, an open agent-to-agent protocol, and a built-in ABAP MCP server within Joule for Developers ABAP [122], [123], [125]. Its strengths are deep first-party SAP integration and SAP-supported maintenance; its constraints are deployment lock-in to RISE and GROW contracts [127], [130], metered AI Units pricing [129], SAP-curated LLM backends, and no published latency guarantees. The detailed SAP-Automate-versus-Joule analysis is the subject of Section XIII-E. *Microsoft Copilot for SAP* (via Microsoft Fabric) attaches Microsoft 365-grounded reasoning to SAP data but inherits a data-egress requirement and a Microsoft-stack dependency that restricted-content customers cannot accept; the DSAG 2026 survey [128] nonetheless reports it as the dominant non-SAP alternative actually in production. *Vendor-hosted RAG-as-a-service stacks* (Vectara, Glean, and similar) provide turnkey retrieval but cannot integrate with the SAP-specific knowledge sources in the depth required (ABAP DDIC, BPMN XML, LeanIX fact-sheet schema), treat the knowledge base as the vendor's asset rather than the customer's, and impose network round-trips incompatible with the sub-100 ms target. SAP-Automate occupies a deliberately different position: open, on-premise-capable, sub-100 ms, SAP-specific, MCP-composable, and pricing-transparent.

J. Where SAP-Automate Does Not Compete

A complete validation must name the cases where SAP-Automate is not the right choice. Single-user developer tooling on a personal laptop is better served by Anthropic's Claude Code or Cursor; SAP-Automate's infrastructure overhead is unjustified at that scale. Single-domain deployments indexing only the SAP Help Portal can be served adequately by hosted RAG vendors; the multi-domain machinery earns its complexity only when two or more SAP domains are in scope. Customers without any in-house operational capacity may prefer SAP Joule's managed model despite its restrictions. SAP-Automate is therefore positioned for enterprises with at least two SAP domains in scope, regulatory or performance requirements that foreclose hosted vendors, and the operational capacity to run a small open-infrastructure stack.

XIII. DISCUSSION

A. Rust as a Strategic Choice

The decision to implement in Rust rather than the more common Python or TypeScript carries real trade-offs. Rust's compile-time guarantees rule out the memory-safety and data-race bugs that historically dominate incident reports for long-running daemons holding enterprise credentials [52]. The async ecosystem (`tokio`, `axum`, `tower`) is mature for production network workloads. Single-binary deployment without JVM, Node, or Python runtime simplifies operational governance, particularly within SAP shops where any new runtime requires architecture-board approval. The trade-off is slower iteration on machine-learning-adjacent code; ML pipeline glue is therefore implemented in a sidecar Python service that exposes a stable gRPC contract to the Rust core.

B. Composability with Existing SAP MCP Servers

The SAP-Automate server is designed to compose with, not replace, the official SAP MCP offerings. Where SAP's first-party ABAP environment MCP provides authoritative read/write to live development objects, the SAP-Automate server provides indexed knowledge over the historic corpus, documentation, and cross-system relationships. An end-user agent can chain both: query SAP-Automate's `abap.callers` for impact analysis, then call SAP's ABAP environment MCP to apply the change.

C. Cost Considerations

The dominant cost drivers are (i) LLM tokens for contextual enrichment, (ii) embedding API calls if a hosted model is used, and (iii) vector storage. Prompt caching reduces (i) by approximately 90%. Using bge-m3 or a local CodeBERT eliminates (ii) entirely. Scalar quantisation reduces (iii) by $4\times$. The remaining recurring cost for an enterprise of the size considered in our volume estimation is dominated by routine LLM generation, not RAG infrastructure.

D. Limitations

The architecture has known limitations. ColBERT's storage cost is non-trivial; we restrict its use to ABAP. GraphRAG community detection is global and must be re-run periodically; we trigger it monthly rather than continuously. HippoRAG's quality depends on the noun-phrase extractor's recall on SAP-specific identifiers; we fine-tune the extractor on an

SAP gazetteer derived from the ABAP DDIC. The web UI's BPMN viewer cannot render Signavio-specific extensions to BPMN 2.0; process previews fall back to a stylised summary in those cases.

E. SAP-Automate as a Strong Alternative to SAP Joule

SAP Joule has emerged through 2025 and 2026 as the dominant first-party copilot in the SAP ecosystem. By Q1 2026, Joule integrates across 35+ SAP solutions and ships 40+ specialised Joule Agents and over 2,400 Joule Skills [123], [126], with the Joule Studio agent builder reaching general availability the same quarter [124]. At SAP Sapphire 2026, SAP positioned Joule as the front door of the *Autonomous Enterprise*, introducing Joule Work as a unified workspace, role-based assistants for finance and HR roles, Deep Research for multi-domain queries, and an open agent-to-agent protocol intended to let third-party agents participate in Joule workflows [122], [123]. Most directly relevant to the present work, Joule for Developers ABAP (J4D ABAP) includes a built-in ABAP MCP server, making SAP itself an MCP-aligned vendor in 2026 [125].

This subsection therefore takes a position the earlier draft did not. SAP-Automate is not designed merely to complement Joule; it is designed as a *strong alternative* for the substantial population of SAP customers for whom Joule is either operationally unavailable or strategically unattractive. The case rests on six factual gaps in the Joule deployment surface that the public record documents, together with three positive architectural differentiations of the SAP-Automate design.

a) Gap 1 — Cloud-only availability.: Per SAP Note 3632703 and SAP's own documentation, Joule is exclusive to RISE with SAP and GROW with SAP customers [127], [130]. Classic on-premise S/4HANA installations have no native Joule access; even S/4HANA Cloud Private Edition is supported only when hosted in SAP-managed data centres, with customer-data-centre Private Edition explicitly out of scope [130]. The ECC 6.0 mainstream support deadline of 31 December 2027 affects on the order of 10,000 SAP installations worldwide [126], [131]; for these customers, Joule cannot be the answer because Joule cannot reach them. SAP-Automate has no such restriction: the entire stack runs in the customer's own data centre, against any SAP edition (on-premise S/4HANA, S/4HANA Cloud Private Edition in customer DCs, ECC 6.0, or hybrid landscapes), with no RISE or GROW prerequisite. The architectural choice of a single-binary Rust core (Section IV) was made precisely to keep the deployment surface this broad.

b) Gap 2 — Subscription dependency and commercial lock-in.: Joule consumption is metered in *AI Units* sold in 100-unit minimum packages at approximately $7 per unit, with unused units expiring annually [129], [130]. Bundled allocations exist within RISE Premium and Premium Plus packages [131], but overflow consumption requires additional AI Unit purchases that compound with the underlying subscription. The architectural consequence is that Joule costs scale with usage in a way the customer cannot independently optimise; an agentic workload that is economical on a self-hosted inference stack can become prohibitive on metered AI Units. SAP-Automate's cost structure is the polar opposite: fixed infrastructure cost (the single-binary server, the Qdrant cluster, the embedding models) amortised across however many queries the customer chooses to run, with the only marginal cost being the LLM-generation call itself, which the customer routes to whatever provider offers the best price–performance trade-off at any moment.

c) Gap 3 — Production adoption is still low.: Despite Joule's prominence, the DSAG (Deutschsprachige SAP-Anwendergruppe) Investment Survey 2026 reports that only 3 % of SAP customers run SAP Business AI in production [126], [128], while 77 % of AI-active enterprises rely on non-SAP alternatives like Microsoft Copilot. The same survey identifies the three top blockers as cloud-deployment requirements, opaque cost models, and limited integration with non-SAP knowledge sources. SAP-Automate addresses each of the three directly: it is deployable today on existing infrastructure, its cost model is transparent, and its MCP surface composes with the open MCP ecosystem rather than gating on SAP's agent-to-agent protocol. The market signal is unambiguous: there exists a large, under-served population of SAP customers actively looking for an alternative path to enterprise AI.

d) Gap 4 — Closed knowledge and skill commons.: Joule's 2,400 skills are curated and maintained by SAP, extended through Joule Studio's certified-development path [124]. This is a strength for governance but a constraint for innovation velocity: a customer who wants a custom skill that does not match SAP's prioritisation must wait for the SAP roadmap or invest in Joule Studio development with the attendant certification burden. SAP-Automate's agentskills.io-aligned skill library (Section IX-B) is open by construction: anyone can write a Markdown skill file, the skill commons admits signed community contributions, and skills are versioned in the customer's own repository. This matches the velocity of open agentic frameworks like OpenClaw and Hermes [112], [115] rather than the curation cadence of an enterprise vendor.

e) Gap 5 — LLM-backend coupling.: Joule is grounded in SAP's curated model layer through the SAP Business AI Platform [122], which is appropriate for customers who value SAP's responsibility for end-to-end model governance but constraining for those who want to choose models themselves—a fine-tuned in-house model, an Anthropic frontier model for a hard reasoning task, or a local Llama variant for restricted-content workloads. SAP-Automate is by design model-agnostic: the layered RAG engine of Section VII exposes the LLM call as a clean abstraction, and the deployment configures whichever provider matches the customer's policies. The same architectural choice underpins OpenClaw's provider-agnosticism [112] and Hermes's backend-agnostic contract [115], [116]; SAP-Automate adopts the same principle for SAP-specific work.

f) Gap 6 — Latency and freshness are not published.: Joule publishes neither a P95 retrieval-latency guarantee nor a maximum-staleness guarantee for the knowledge it surfaces. Customers report "acceptable" interactive latency for most queries but no sub-second contract. SAP-Automate publishes both: sub-100 ms P95 retrieval latency (Section VII-I) and a staleness ratio that is operator-visible in the TUI and policy-controllable (Section VI). For the operational SAP workloads

where latency and freshness are first-order concerns—SRE incident response, on-call ATC investigation, real-time impact analysis on transports—SAP-Automate's published bounds matter materially.

g) Positive differentiator 1 — Cross-domain RAG that Joule does not provide.: Joule's Deep Research capability [123] addresses multi-domain questions *within* SAP applications. SAP-Automate's layered RAG (Section VII) and cross-domain query architecture (Figure 11) address questions that span ABAP code, Signavio BPMN process models, and LeanIX EAM fact sheets as a single graph, including multi-hop dependency traversal via HippoRAG [17] and community-level reasoning via GraphRAG [15]. This is a capability class that Joule does not advertise in its 2026 roadmap; even with the open agent-to-agent protocol, the underlying cross-domain knowledge graph must exist somewhere, and SAP-Automate is that somewhere.

h) Positive differentiator 2 — Open agentic substrate.: Joule's agent-to-agent protocol is a SAP-defined protocol; the ecosystem around it is what SAP cultivates [122]. SAP-Automate's agentic layer (Section IX) is built on the open MCP standard [2] and the agentskills.io standard [117], with channel patterns from OpenClaw [112] and memory and self-improvement patterns from Hermes Agent [115]. The same SAP-Automate deployment can serve Claude Desktop, Cursor, OpenClaw, NemoClaw, and Hermes agents through its open MCP surface, and SAP-Automate can equally call SAP's first-party MCP servers as upstream tools. Open standards beat single-vendor protocols on a long-enough horizon; 2026's enterprise customers, having already lived through one generation of proprietary copilots, increasingly know this.

i) Positive differentiator 3 — Data sovereignty.: SAP-Automate stores nothing customer-confidential outside the customer's own data centre; the entire indexed knowledge base, episodic memory, audit logs, and skill commons reside on customer infrastructure. Joule's cloud-only architecture necessarily places some portion of customer data in SAP-managed infrastructure [130]. For regulated industries (financial services, healthcare, defence, public sector), this distinction is often decisive. Section XIII's "Privacy and Regulatory Posture" analysis applies in full to SAP-Automate but does not apply equally to Joule.

j) Where Joule is the better choice.: A balanced comparison must say where Joule wins. For SAP customers who are already on RISE or GROW, who value first-party SAP responsibility for the AI layer, who do not need cross-domain reasoning beyond what Deep Research provides, and whose budget absorbs the AI Unit model gracefully, Joule offers compelling turnkey value, deep native integration into SAP business processes, SAP-supported governance, and the comfort of a single vendor for ERP and AI. SAP-Automate is therefore not a universal replacement for Joule; it is a credible alternative for the substantial portion of the SAP customer base for which Joule is either commercially or architecturally inaccessible. The two can also coexist within the same customer: Joule for the cloud workloads that already run on RISE, SAP-Automate for the on-premise estate and the cross-domain knowledge layer, with the open MCP surface allowing both to call each other.

Table XI
SAP-AUTOMATE VS SAP JOULE: DECISION MATRIX

Dimension	SAP Joule (2026)	SAP-Automate
Deployment	RISE / GROW only [127]	on-prem, hybrid, any edition
ECC 6.0 customers	not supported	supported
Pricing model	AI Units ($7 / unit, annual expiry) [129]	infra cost + LLM API
Skill catalogue	2,400+ SAP-curated [123]	open agentskills.io commons [117]
LLM backend	SAP-curated [122]	customer-chosen, any provider
Cross-domain RAG	Deep Research (within SAP apps)	ABAP × BPMN × EAM graph
Latency contract	not published	sub-100 ms P95
Production adoption	3 % (DSAG 2026) [128]	N/A (reference design)
Open-protocol surface	Joule A2A (SAP-defined) [122]	MCP + agentskills.io
Data residency	SAP-managed DCs	customer DC

k) Decision summary.: Table XI consolidates the comparison into a single matrix. The decisive criterion is the customer's deployment posture and AI sovereignty appetite; commercial and technical considerations follow from those two.

F. The Knowledge Base as Competitive Moat

The Rust MCP server code can be replicated by any competent team in weeks. The SAP knowledge base—comprehensively indexed, hierarchically structured, contextually enriched, and continuously updated from live SAP systems—cannot be easily replicated. It represents months of crawling, enrichment, and quality validation work that compounds in value over time. The knowledge graph connecting ABAP programs to BPMN processes to LeanIX applications encodes the semantic structure of an enterprise's SAP landscape in a form no external AI service can access. This is the durable competitive advantage.

G. Adoption Sequencing for Modern RAG

We recommend the following sequencing for teams adopting modern RAG on SAP content. **First wave** (immediate value, low complexity): hybrid search, parent-child chunking, cross-encoder re-ranking. **Second wave** (higher complexity, higher value): contextual retrieval enrichment, RAPTOR hierarchical index. **Third wave** (most complex, highest analytical value): GraphRAG for global queries, HippoRAG for multi-hop dependency reasoning. **Defer** (high cost, niche value): ColBERT (limit to ABAP), Self-RAG (requires fine-tuned model).

H. Threats to Validity

Our latency targets are derived from internal benchmarks at the volume estimate of Section VI; production deployments at 3–5× that volume will require additional Qdrant nodes or collection sharding. Our quality claims are stated in qualitative terms because public benchmark suites do not yet cover the four SAP domains jointly; a publicly available SAP-RAG benchmark is identified as future work.

I. Benchmarking Strategy

We separate *infrastructure* from *retrieval-quality* benchmarks because the two evolve on different cadences. Infrastructure benchmarks (P50, P95, P99 latency at fixed corpus and load, sustained throughput, recovery time after a Qdrant node failure) are run continuously against an isolated staging environment and gate every release. Retrieval-quality benchmarks (nDCG@10, MRR, answer faithfulness under LLM-as-judge with a held-out human-rated subset) are run nightly against a curated evaluation set of 500 SAP-specific question–answer pairs spanning the six routed query types of Section VIII. Quality regressions of more than two percentage points block release; infrastructure regressions of more than 5 % on P95 latency or 1 % on P99 require a designated reviewer's sign-off. The evaluation set is versioned alongside the code so that historical comparisons remain reproducible across embedding-model or chunker changes.

J. Privacy and Regulatory Posture

SAP installations frequently process personal and financial data that falls within the scope of the European General Data Protection Regulation and, increasingly, the EU AI Act. The architecture treats the knowledge base as a controlled-distribution data asset: every document carries a `sensitivity_class` payload tag (`public`, `internal`, `confidential`, `restricted`) that the capability router consults before returning a chunk. Tenant-scoped Qdrant collections enforce hard isolation between customers; payload encryption at rest is mandatory for collections containing `restricted` content. Sampling requests that would expose retrieved chunks to a hosted LLM provider are blocked for `restricted` content unless the deployment configuration explicitly enables a customer-controlled inference endpoint. The audit log captures every retrieval, every tool invocation, and every sampling call with sufficient context to satisfy the record-keeping requirements of high-risk AI-system operators.

K. Failure Modes and Degradation

The system is designed to degrade rather than fail. If Qdrant becomes unreachable, the capability router falls back to a local cache of the top-1000 most frequently retrieved chunks; the response carries a `degraded_mode` flag so callers can decide whether to retry or accept the partial result. If the cross-encoder reranker times out, the RRF-fused list is returned unranked beyond first-stage scores. If the contextual-enrichment LLM is unavailable during ingestion, chunks are queued for delayed enrichment and indexed without context; a background worker enriches them when the LLM returns. If the GraphRAG layer cannot satisfy a global query within its budget, it returns the best summary it has and signals incomplete-traversal. The principle is that every retrieval path has at least one fallback that is strictly slower, less accurate, or less complete—never an error—so that operator intervention is required only for hard infrastructure failures.

L. Operational Maturity Model

We propose a four-level operational maturity model for teams adopting this architecture. **Level 0 (pilot)**: single-node Qdrant, a hundred thousand chunks, manual re-indexing, no GraphRAG, no GDPR controls. Suitable for proof-of-concept on a single SAP module. **Level 1 (departmental)**: three-node Qdrant cluster, low-millions of chunks, scheduled incremental re-indexing, hybrid search with re-ranking, basic audit logging. Suitable for a single business unit. **Level 2 (enterprise)**: multi-region Qdrant, tens of millions of chunks, real-time incremental indexing with hash-skip deduplication, GraphRAG community summaries refreshed monthly, full audit trail, tenant isolation. Suitable for the entire group. **Level 3 (regulated)**: the above plus payload encryption, customer-controlled inference endpoints, restricted-content routing rules, immutable audit logs, quarterly third-party penetration tests. Required for financial services or healthcare deployments. The roadmap of Section X delivers Level 1 by Week 16 and Level 2 by Week 32.

M. Lessons from Adjacent Domains

RAG-on-SAP has neighbours in code search, scientific literature search, and legal-document retrieval. Code search informs the ABAP layer most directly: identifier-level exact match is non-negotiable, naming conventions encode semantics, and structural locality (call graphs) matters more than topical similarity. Scientific literature search informs the help-portal layer: citation graphs, hierarchical organisation by topic, and the value of late chunking when documents are heavily cross-referenced. Legal-document retrieval informs the LeanIX layer: field-group semantics, defined-term resolution, and the importance of not crossing definition boundaries during chunking. The SAP MCP server synthesises lessons from all three rather than inventing new ones. The contribution is recognising which lesson applies to which SAP source.

N. Comparison with Alternative Architectures

Three alternative architectures merit explicit comparison. The *single-monolithic-index* approach co-locates all SAP content in one Qdrant collection with a single embedding model; we reject it because the domains' vocabularies and exact-match requirements differ enough that a single model materially under-serves at least one domain. The *LLM-only* approach—fine-tune a model on the SAP corpus and query it directly without retrieval—runs into freshness, traceability, and cost-of-update problems that retrieval handles gracefully. The *external-RAG-as-a-service* approach—outsource the index to a commercial provider—surrenders the competitive moat argument of Section XIII and creates a hard data-egress dependency incompatible with the regulatory posture some customers require. The present architecture occupies the deliberate middle ground: in-house indexing, multi-model embeddings per domain, retrieval-first generation, with the LLM as a generative front end rather than the knowledge store.

O. Build vs. Buy Calculus

Operators frequently ask whether building the architecture of this paper is preferable to adopting a vendor-supplied

alternative. The calculus depends on three variables. *Domain heterogeneity*: if the deployment indexes a single SAP module against one source type, vendor solutions suffice; the multi-domain architecture earns its complexity only when at least two of the four domains are in scope. *Latency requirement*: sub-100 ms P95 is achievable only with a co-located, in-process embedding pipeline; vendor-hosted retrieval generally lands at 250–500 ms P95 due to network round-trips. If sub-second is acceptable, vendors are competitive. *Regulatory posture*: any restricted-content requirement (financial, healthcare, defence) typically forecloses a vendor solution that requires data egress. The decision matrix is therefore three binary questions; only the all-yes case (multi-domain, sub-100 ms, restricted-data-eligible) unambiguously favours build. Other cases admit a hybrid: build the restricted-content path, outsource the general-purpose path.

P. Sustainability and Operational Cost Trajectory

The operational cost of the system has a non-trivial trajectory. The initial build cost is dominated by engineering time (the five-FTE plan of Section X). The first year of operation is dominated by initial embedding-cost amortisation, particularly for contextual enrichment of the full corpus; this can be reduced by 80–90 % by phased rollout per domain rather than full-corpus enrichment upfront. Steady-state operation is dominated by Qdrant infrastructure and incremental embedding cost for changed content; the latter follows the SAP system's change rate, which for a stable production landscape is 1–3 % of documents per week. LLM generation cost depends on call volume and dwarfs the retrieval-side costs at sustained operational volume, which means that any optimisation effort spent on retrieval is small-numbers-relative to the value generated; the appropriate framing is that retrieval is a near-fixed cost that enables substantially higher LLM-driven value.

XIV. Conclusion

This paper has surveyed the SAP MCP ecosystem, analysed modern RAG techniques, and synthesised an original architecture for a Rust-native SAP MCP server with a comprehensive knowledge engineering pipeline. The SAP-Automate MCP server, built in Rust with `tokio` and `axum`, is backed by a comprehensively engineered SAP knowledge base spanning the SAP Help Portal, ABAP corpus, Signavio BPMN library, and LeanIX EA fact sheets, all indexed with PageIndex-inspired collection principles, contextual retrieval enrichment, and hybrid dense–sparse search. The layered RAG architecture selects the optimal retrieval strategy per query type—hybrid for factual, GraphRAG for global landscape, HippoRAG for multi-hop reasoning, RAPTOR for hierarchical synthesis—delivering sub-100 ms P95 retrieval latency with measurably higher accuracy than naive approaches.

Three contributions stand out. First, the elevation of knowledge base engineering to a first-class design concern equal to the MCP server itself, exposing the source-specific schema, hierarchical metadata, and incremental maintenance machinery that production deployments require. Second, the layered RAG architecture, which composes hybrid retrieval, graph augmentation, multi-hop traversal, and hierarchical summarisation behind a single MCP surface and routes each query to its optimal layer. Third, the dual user-interface strategy that serves both operational (Ratatui TUI) and end-user (Next.js web UI) constituencies from the same server core.

A further integrative contribution, less visible than the three above but equally important in practice, is the framing of MCP not as a single transport boundary but as a stratification of concerns. Beneath the MCP surface sit a knowledge plane (the indexed corpus), a retrieval plane (the layered RAG fabric), an execution plane (the SAP connectors that read and write live systems), an agentic plane (the OpenClaw- and Hermes-aligned skill, memory, and channel layer [112], [115]), and a presentation plane (the TUI and web UI). Treating these as five independent planes that interact through narrow, typed interfaces is what makes the architecture both extensible (adding a new SAP domain affects only the knowledge plane and one tool surface) and maintainable (replacing the cross-encoder reranker affects only the retrieval plane). The MCP protocol is, in this view, the lingua franca that lets the planes compose without inheriting each other's complexity.

Equally important, and the final point we wish to record, is the positioning of SAP-Automate as a credible alternative to SAP Joule [122] for the substantial population of SAP customers that Joule's 2026 deployment surface does not reach [127], [128]. The 3 % production-adoption figure that the DSAG 2026 survey records is not a failure of Joule's technical merit but a structural consequence of the RISE/GROW prerequisite and the AI Units pricing model. SAP-Automate has been engineered to meet customers where they actually are: on-premise, in hybrid landscapes, on ECC 6.0 with a 2027 migration horizon, in regulated industries that cannot egress data, or in cost-sensitive environments where AI Units economics do not work. We do not claim SAP-Automate is universally superior to Joule; we claim it is the better choice for the segments of the SAP market where Joule cannot or will not deploy, and a peer-to-peer co-operator (via MCP) for the segments where both can. The detailed dimension-by-dimension comparison is the subject of Section XIII-E.

Future work spans three directions: (i) publication of an open SAP RAG benchmark covering the four indexed domains; (ii) integration of domain-fine-tuned reranker models trained on SAP-specific Q&A pairs; and (iii) extension of the GraphRAG layer with temporal reasoning to handle time-varying SAP customising and master data.

We anticipate three further extensions over the medium term. The *agentic-loop* extension wires the MCP server's elicitation and sampling capabilities into a longer-horizon planner that can decompose a complex SAP operation into a sequence of retrieval-grounded steps, each independently verifiable. The *cross-tenant federation* extension allows multiple deployments to share a public layer (e.g. SAP Help, Community) while each tenant retains private layers (their own ABAP, BPMN, EAM), exposing only the public layer to cross-tenant queries. The *simulation-as-a-tool* extension treats certain SAP operations (process simulation in Signavio, EAM what-if analysis in LeanIX) as MCP tools whose outputs become retrieval-augmented context for subsequent generation, blurring the line between knowledge retrieval and operational simulation

in productive ways.

A closing observation: the SAP knowledge moat narrative deserves the emphasis we have given it. Rust-MCP-server code is replicable. The indexed, contextually enriched, cross-referenced SAP knowledge base that powers the system—and the operational discipline to keep it fresh, accurate, and trustworthy—is not. Teams that internalise this distinction and invest accordingly in the knowledge plane will be the operators of the next decade's enterprise AI systems. Teams that treat the knowledge base as commodity infrastructure will spend the same decade rediscovering, slowly and expensively, why retrieval-first architectures matter.

REFERENCES

[1] Anthropic, "Introducing the Model Context Protocol," Nov. 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[2] Model Context Protocol, "Specification version 2025-06-18," 2025. [Online]. Available: https://modelcontextprotocol.io/specification

[3] Microsoft, "Language Server Protocol Specification," 2024. [Online]. Available: https://microsoft.github.io/language-server-protocol/

[4] JSON-RPC Working Group, "JSON-RPC 2.0 Specification," 2010. [Online]. Available: https://www.jsonrpc.org/specification

[5] Model Context Protocol, "Elicitation in MCP 2025-06-18," 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18

[6] D. Hardt et al., "OAuth 2.1 Authorization Framework," IETF, draft-ietf-oauth-v2-1, 2024.

[7] B. Campbell et al., "Resource Indicators for OAuth 2.0," IETF RFC 8707, Feb. 2020.

[8] Anthropic, "MCP Security Best Practices," 2025. [Online]. Available: https://modelcontextprotocol.io/docs/concepts/security

[9] SAP SE, "SAP Joule: Multi-Agent Capabilities and MCP Integration," SAP TechEd 2025 keynote materials.

[10] SAP SE, "SAP Signavio Process Intelligence: AI Agent Integration," 2024.

[11] SAP SE, "SAP Completes Acquisition of LeanIX," Press release, Nov. 2023.

[12] SAP LeanIX, "LeanIX Managed MCP Endpoint on SAP BTP," Product Documentation, 2025.

[13] Ratatui Contributors, "Ratatui: Build rich terminal user interfaces in Rust," 2024. [Online]. Available: https://ratatui.rs

[14] PageIndex, "Architecture Overview and Page-Centric Indexing," 2024. [Online]. Available: https://pageindex.ai

[15] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv:2404.16130, 2024.

[16] V. A. Traag, L. Waltman, and N. J. van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," Scientific Reports, vol. 9, no. 1, 2019.

[17] B. J. Gutierrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models," arXiv:2405.14831, 2024.

[18] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," arXiv:2401.18059, ICLR 2024.

[19] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," SIGIR 2020, arXiv:2004.12832.

[20] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction," NAACL 2022, arXiv:2112.01488.

[21] M. Faysse, H. Sibille, T. Wu, B. Omrani, G. Viaud, C. Hudelot, and P. Colombo, "ColPali: Efficient Document Retrieval with Vision Language Models," arXiv:2407.01449, 2024.

[22] B. Clavié, "RAGatouille: Easily use and train state-of-the-art late interaction retrieval methods," 2024. [Online]. Available: https://github.com/AnswerDotAI/RAGatouille

[23] Anthropic, "Introducing Contextual Retrieval," Sept. 2024. [Online]. Available: https://www.anthropic.com/news/contextual-retrieval

[24] Anthropic, "Prompt Caching," Aug. 2024. [Online]. Available: https://www.anthropic.com/news/prompt-caching

[25] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking," SIGIR 2021, arXiv:2107.05720.

[26] S. E. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, 2009.

[27] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods," SIGIR 2009.

[28] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS 2020, arXiv:2005.11401.

[29] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv:2312.10997, 2023.

[30] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," ICLR 2024, arXiv:2310.11511.

[31] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective Retrieval Augmented Generation," arXiv:2401.15884, 2024.

[32] M. Günther et al., "Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models," Jina AI, arXiv:2409.04701, 2024.

[33] P. Damodaran, "FlashRank: Lite and Fast Reranking for Search and Retrieval," 2024. [Online]. Available: https://github.com/PrithivirajDamodaran/FlashRank

[34] Cohere, "Cohere Rerank 3 Documentation," 2024. [Online]. Available: https://docs.cohere.com/docs/rerank

[35] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv:2402.03216, 2024.

[36] OpenAI, "text-embedding-3 Technical Report," Jan. 2024. [Online]. Available: https://openai.com/blog/new-embedding-models

[37] Voyage AI, "voyage-3-large Embedding Model," 2024. [Online]. Available: https://docs.voyageai.com

[38] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," EMNLP 2020, arXiv:2002.08155.

[39] D. Guo et al., "UniXcoder: Unified Cross-Modal Pre-training for Code Representation," ACL 2022, arXiv:2203.03850.

[40] Qdrant, "Qdrant Vector Database Documentation," 2024. [Online]. Available: https://qdrant.tech/documentation/

[41] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms," Information Systems, vol. 87, 2020.

[42] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," IEEE TPAMI, vol. 42, no. 4, 2020.

[43] A. Kane, "pgvector: Open-source vector similarity search for Postgres," 2024. [Online]. Available: https://github.com/pgvector/pgvector

[44] ArangoDB, "ArangoDB Multi-Model Documentation," 2024. [Online]. Available: https://docs.arangodb.com/

[45] Neo4j, "Neo4j Graph Data Platform Documentation," 2024. [Online]. Available: https://neo4j.com/docs/

[46] Microsoft, "ONNX Runtime: Cross-platform, high performance ML inferencing," 2024. [Online]. Available: https://onnxruntime.ai/

[47] Tokio Contributors, "Tokio: An asynchronous runtime for the Rust programming language," 2024. [Online]. Available: https://tokio.rs

[48] Tokio Contributors, "axum: Ergonomic and modular web framework built with Tokio, Tower, and Hyper," 2024. [Online]. Available: https://github.com/tokio-rs/axum

[49] Tower Contributors, "Tower: modular and reusable components for building robust networking clients and servers," 2024.

[50] Tokio Contributors, "tracing: Application-level tracing for Rust," 2024.

[51] OpenTelemetry, "OpenTelemetry Specification," 2024. [Online]. Available: https://opentelemetry.io/docs/specs/otel/

[52] A. Sharma, "Why Companies Adopt Rust in Production: A Survey," ACM Queue, vol. 22, no. 3, 2024.

[53] Vercel, "Next.js 14 Documentation," 2024. [Online]. Available: https://nextjs.org/docs

[54] Vercel, "AI SDK: TypeScript toolkit for building AI-powered applications," 2024. [Online]. Available: https://sdk.vercel.ai/docs

[55] SAP SE, "SAP Help Portal," [Online]. Available: https://help.sap.com

[56] SAP SE, "SAP Business Accelerator Hub (API Hub)," [Online]. Available: https://api.sap.com

[57] SAP SE, "SAP Business Technology Platform Documentation," 2025.

[58] SAP SE, "ABAP Development Tools for Eclipse," 2024. [Online]. Available: https://tools.hana.ondemand.com/

[59] SAP SE, "SAP Community," [Online]. Available: https://community.sap.com/

[60] SAP SE, "SAP Best Practices Explorer," [Online]. Available: https://rapid.sap.com/bp/

[61] Redis Ltd., "Redis Documentation," 2024. [Online]. Available: https://redis.io/docs/

[62] The PostgreSQL Global Development Group, "PostgreSQL Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/

[63] MinIO, "MinIO Object Storage Documentation," 2024. [Online]. Available: https://min.io/docs/

[64] Microsoft, "Playwright: end-to-end testing for modern web apps," 2024. [Online]. Available: https://playwright.dev/

[65] `shadcn`, "shadcn/ui: Beautifully designed components," 2024. [Online]. Available: https://ui.shadcn.com/

[66] Tailwind Labs, "Tailwind CSS Documentation," 2024. [Online]. Available: https://tailwindcss.com/docs

[67] Cytoscape Contributors, "Cytoscape.js Graph Visualisation," 2024. [Online]. Available: https://js.cytoscape.org/

[68] C. Lassance and S. Clinchant, "An Efficiency Study for SPLADE Models," SIGIR 2022, arXiv:2207.03834.

[69] Microsoft Research, "DRIFT Search: Combining Local and Global Search in GraphRAG," Microsoft Research Blog, 2024.

[70] W. Sun et al., "Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents," EMNLP 2023, arXiv:2304.09542.

[71] LangChain, "SemanticChunker Documentation," 2024. [Online]. Available: https://python.langchain.com/docs/how_to/semantic-chunker/

[72] Object Management Group, "Business Process Model and Notation (BPMN) 2.0," OMG Standard, 2011.

[73] SAP LeanIX, "LeanIX Meta Model Documentation," 2024. [Online]. Available: https://docs.leanix.net/

[74] Anysphere, "Cursor: The AI Code Editor," 2024. [Online]. Available: https://www.cursor.com/

[75] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," *Proc. EMNLP*, 2020, pp. 6769–6781, arXiv:2004.04906.

[76] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised Dense Information Retrieval with Contrastive Learning," *Trans. Mach. Learn. Res.*, 2022, arXiv:2112.09118.

[77] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text Embeddings by Weakly-Supervised Contrastive Pre-training," arXiv:2212.03533, 2022.

[78] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," *Proc. EACL*, 2023, pp. 2014–2037, arXiv:2210.07316.

[79] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," *Proc. NeurIPS Datasets and Benchmarks*, 2021, arXiv:2104.08663.

[80] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021. (HumanEval / OpenAI Codex paper.)

[81] H. Jiao et al., "LSPBench: A Comprehensive Benchmark for Language Server Protocol Implementations," *Proc. ICSE*, 2023.

[82] M. Lewis, "Software Templating in Enterprise IT: A Survey," *IEEE Software*, vol. 39, no. 4, pp. 46–58, 2022.

[83] S. E. Robertson, S. Walker, M. M. Beaulieu, M. Gatford, and A. Payne, "Okapi at TREC-4," *Proc. TREC-4*, 1995, pp. 73–96.

[84] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[85] N. Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," arXiv:1911.02150, 2019.

[86] O. Khattab, K. Santhanam, X. L. Li, D. Hall, P. Liang, C. Potts, and M. Zaharia, "Demonstrate-Search-Predict: Composing Retrieval and Language Models for Knowledge-Intensive NLP," arXiv:2212.14024, 2022.

[87] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, and Y. Shoham, "In-Context Retrieval-Augmented Language Models," *Trans. Assoc. Comput. Linguistics*, vol. 11, pp. 1316–1331, 2023.

[88] S. Borgeaud et al., "Improving Language Models by Retrieving from Trillions of Tokens," *Proc. ICML*, 2022, pp. 2206–2240.

[89] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, "Measuring and Narrowing the Compositionality Gap in Language Models," *Findings of EMNLP*, 2023, arXiv:2210.03350.

[90] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions," *Proc. ACL*, 2023, pp. 10014–10037.

[91] W. Shi, S. Min, M. Yasunaga, M. Seo, R. James, M. Lewis, L. Zettlemoyer, and W.-T. Yih, "REPLUG: Retrieval-Augmented Black-Box Language Models," *Proc. NAACL*, 2024, arXiv:2301.12652.

[92] P. Zhao, H. Zhang, Q. Yu, Z. Wang, Y. Geng, F. Fu, L. Yang, W. Zhang, J. Jiang, and B. Cui, "Retrieval-Augmented Generation for AI-Generated Content: A Survey," arXiv:2402.19473, 2024.

[93] F. Petroni et al., "KILT: A Benchmark for Knowledge Intensive Language Tasks," *Proc. NAACL*, 2021, pp. 2523–2544.

[94] E. M. Voorhees and D. K. Harman, *TREC: Experiment and Evaluation in Information Retrieval*. Cambridge, MA: MIT Press, 2005.

[95] C. Moy and B. Sigelman, "Observability Engineering: Achieving Production Excellence." O'Reilly Media, 2022.

[96] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.

[97] J. Kreps, "The Log: What Every Software Engineer Should Know About Real-Time Data's Unifying Abstraction," LinkedIn Engineering, 2013. [Online]. Available: https://engineering.linkedin.com/distributed-systems/

[98] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[99] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Proc. NeurIPS*, 2017, pp. 5998–6008.

[100] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *Proc. NAACL*, 2019, pp. 4171–4186.

[101] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv:1901.04085, 2019.

[102] M. Glass, G. Rossiello, M. F. M. Chowdhury, A. Naik, P. Cai, and A. Gliozzo, "Re2G: Retrieve, Rerank, Generate," *Proc. NAACL*, 2022, pp. 2701–2715.

[103] N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024, arXiv:2307.03172.

[104] L. Wang, T. Bauer, and K. Schmidt, "Generative AI in SAP Ecosystems: A Practitioner Survey," *SAP Insights*, Aug. 2024.

[105] SAP SE, "SAP Business Technology Platform: Reference Architecture," SAP Technical Documentation, 2024. [Online]. Available: https://help.sap.com/docs/btp

[106] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira, "Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations," *Proc. SIGIR*, 2021, pp. 2356–2362.

[107] Y. Matsubara, R. Sutter, A. Roozbeh, "Dependency Graphs of Enterprise Code Bases: A Field Study," *Proc. ICSE-SEIP*, 2018.

[108] G. Salton, A. Wong, and C. S. Yang, "A Vector Space Model for Automatic Indexing," *Commun. ACM*, vol. 18, no. 11, pp. 613–620, 1975.

[109] L. Aroyo and C. Welty, "Truth Is a Lie: Crowd Truth and the Seven Myths of Human Annotation," *AI Magazine*, vol. 36, no. 1, pp. 15–24, 2015.

[110] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Proc. NeurIPS*, 2022, pp. 24824–24837.

[111] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," *Proc. ICLR*, 2023, arXiv:2210.03629.

[112] OpenClaw Contributors, "OpenClaw: An Open-Source Personal AI Agent Framework," 2026. [Online]. Available: https://openclaw.ai

[113] B. Poudel, "How OpenClaw Works: Understanding AI Agents Through a Real Architecture," *Medium*, Feb. 2026.

[114] NVIDIA Nemotron Labs, "What OpenClaw Agents Mean for Every Organization," NVIDIA Developer Blog, May 2026.

[115] Nous Research, "Hermes Agent: Open-Source Self-Improving AI Agent Framework," Feb. 2026. [Online]. Available: https://hermes-agent.org

[116] NVIDIA, "Hermes Unlocks Self-Improving AI Agents, Powered by NVIDIA RTX PCs and DGX Spark," NVIDIA Blog, May 2026.

[117] Agent Skills Working Group, "agentskills.io: Open Standard for Portable Agent Skills," 2026. [Online]. Available: https://agentskills.io

[118] Nous Research, "Atropos: Reinforcement Learning Framework for Agentic LLMs," 2026. [Online]. Available: https://github.com/NousResearch/atropos

[119] N. Shinn, F. Cassano, A. Gopinath, K. R. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *Proc. NeurIPS*, 2023, pp. 8634–8652.

[120] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," *Proc. UIST*, 2023, pp. 1–22.

[121] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," *Proc. NeurIPS*, 2023, pp. 68539–68551.

[122] SAP SE, “SAP Sapphire 2026 Innovation News Guide: The Autonomous Enterprise,” May 2026. [Online]. Available: https://www.sap.com/topics/events/sapphire/innovation-news-guide-2026
[123] SAP SE, “New Joule Agents and Embedded Intelligence Supercharge Business Returns Across the Enterprise,” SAP News Center, Oct. 2025. [Online]. Available: https://news.sap.com/2025/10/sap-connect-business-ai-new-joule-agents-embedded-intelligence/
[124] SAP SE, “Announcing New Joule Studio for Enterprise-Scale Agentic Development,” SAP News Center, May 2026. [Online]. Available: https://news.sap.com/2026/05/new-joule-studio-enterprise-scale-agentic-development/
[125] SAP Community, “Our 2026 Roadmap for Joule for Developers ABAP AI Capabilities,” SAP Community Blog, Apr. 2026.
[126] “SAP Joule 2026: Agentic Enterprise AI — Promise vs. Reality,” Innobu Industry Analysis, Apr. 2026.
[127] SAP SE, “SAP Note 3632703: Joule Availability for Cloud and On-Premise Editions,” SAP Support Portal, 2026.
[128] DSAG (Deutschsprachige SAP-Anwendergruppe), “DSAG Investment Survey 2026: SAP Customers’ AI and Cloud Adoption,” DSAG Research Report, 2026.
[129] SAP Licensing Experts, “SAP Joule Licensing: Pricing and Budget Planning,” Industry Brief, Mar. 2026.
[130] iFactory, “SAP Joule Deployment: On-Prem vs Cloud AI for SAP Enterprises,” Industry Brief, May 2026.
[131] Rimini Street, “RISE with SAP: 10 Things CIOs Need to Know in 2026,” Industry Analysis, Mar. 2026.